Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Chapter 7
Fill-reducing orderings

The fill-minimization problem can be stated as follows. *Given a matrix A, find a row and column permutation P and Q (with the added constraint that $Q = P^T$ for a sparse Cholesky factorization) such that the number of nonzeros in the factorization of PAQ, or the amount of work required to compute it, are minimized.* The problem is impossible to solve in practice, so heuristics are used that attempt to reduce fill-in. Three basic strategies exist; they are sometimes combined to obtain hybrid strategies: (1) minimum degree and its variants (minimum fill, for example), (2) nested dissection (recursive graph partitioning), and (3) band reduction. The first strategy is most common and is presented in detail below. Features of the other two are highlighted. The Dulmage–Mendelsohn decomposition is a permutation that reduces the work required for LU and QR factorization. It consists of two primary steps: a permutation to obtain a zero-free diagonal and another to permute the matrix into block triangular form. The method is also useful for converting an edge separator into a node separator, which is used in many nested dissection methods.

7.1 Minimum degree ordering

The *minimum degree* algorithm is a widely used heuristic for finding a permutation P so that PAP^T has fewer nonzeros in its factorization than A. It is a greedy method that selects the sparsest pivot row and column during the course of a right-looking sparse Cholesky factorization (see Section 4.9). Consider the following MATLAB fragment, which does no pivoting:

```
for k = 1:n
    L (k,k) = sqrt (A (k,k)) ;
    L (k+1:n,k) = A (k+1:n,k) / L (k,k) ;
    A (k+1:n,k+1:n) = A (k+1:n,k+1:n) - L (k+1:n,k) * L (k+1:n,k)' ;
end
```

The `k`th step updates `A` with the outer product `L(:,k)*L(:,k)'`. Let $A^{[k]}$ denote the matrix `A(k:n,k:n)` at the start of the `k`th iteration, above.[11] Consider the

[11] This use of the $A^{[k]}$ notation differs from its use in Chapter 5, in which $A^{[k]} = H_k \dots H_1 A$.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

undirected graph of $A^{[k]}$, $G^{[k]} = (V, E)$ with nodes $V = \{k \dots n\}$ and $(i, j) \in E$ if $a_{ij}^{[k]} \neq 0$. The graph $G^{[k]}$ is called the *elimination graph*. If $\mathcal{L}_k$ is the nonzero pattern of `L(:,k)`, the update to $G^{[k]}$ and $A^{[k]}$ corresponds to adding a dense submatrix to `A`, and adding a clique in $G^{[k]}$ and removing node k. Rather than representing $A^{[k]}$ as a simple graph, a *quotient graph* $\mathcal{G}^{[k]}$ represents this clique implicitly. Node k is replaced with *element* k with neighbors $\mathcal{L}_k$. The term "element" is borrowed from finite-element methods, since they too are a collection of dense submatrices or cliques in a graph. The adjacency list of an uneliminated node $i > k$ has two kinds of nodes adjacent to it: $\mathcal{A}_i$ is a list corresponding to original nonzeros a_{ij}, and $\mathcal{E}_i$ is a list of the elements adjacent to i. The nonzero pattern of row or column i is thus

$$\left(\mathcal{A}_i \cup \bigcup_{e \in \mathcal{E}_i} \mathcal{L}_e \right) \setminus \{i\}, \tag{7.1}$$

excluding the diagonal (no self-edges occur in G or $\mathcal{G}$). The degree d_i of node i is the size of the set (7.1). When node k is eliminated, any elements adjacent to it are no longer required to represent the nonzero pattern of $A^{[k]}$ (a consequence of Theorem 4.13); these elements can be removed (called *element absorption*). An example sequence of graphs G and quotient graphs $\mathcal{G}$ is given in Figure 7.1. In the graphs, a plain circle represents a node in $\mathcal{G}$, while a dark circle represents an element. In the matrices, a filled-in circle represents an edge between two nodes in $\mathcal{G}$, a circle is an edge no longer in G, and a circled x is an edge in G represented by an element in $\mathcal{G}$.

Additional terms in $\mathcal{G}$ can be pruned. If two nodes i and j are both in the pivotal element $\mathcal{L}_k$, then j and i can be removed from $\mathcal{A}_i$ and $\mathcal{A}_j$, respectively. They may have been adjacent due to an original entry a_{ij} and are still adjacent in $\mathcal{G}$ because they are both adjacent to element k. That is, $\mathcal{A}_i$ can be replaced with the smaller set $\mathcal{A}_i \setminus \mathcal{L}_k$ for all $i \in \mathcal{L}_k$ (referred to here as *pruning*). With element absorption and pruning of the $\mathcal{A}_i$ sets, the graph $\mathcal{G}$ can be represented in place (its size never exceeds $|A|$).

With this graph representation $\mathcal{G}$, the minimum degree algorithm consists simply of a greedy reordering of the nodes. Rather than selecting node k at the kth step, the algorithm selects the node with the least degree. When an element k is created, the degree of all nodes $i \in \mathcal{L}_k$ must be recomputed, using (7.1). This is the most costly part of the algorithm.

The cost can be reduced by exploiting supernodes, also called *indistinguishable nodes*. If two nodes i and j become identical ($\mathcal{E}_i = \mathcal{E}_j$ and $\mathcal{A}_i = \mathcal{A}_j$), they will remain identical until one of them is eliminated (either both are adjacent to the pivotal element or both are not adjacent). When one of them becomes the node of least degree, the other node is also a node of least degree, and eliminating both causes no more edges in $\mathcal{G}$ than when eliminating just one of them. Thus, if two nodes i and j are found to be indistinguishable, they can be merged into a single supernode that represents both of them. This is done by removing one of the nodes (j, say) and letting i be a representative of the supernode containing both i and j (j has been *absorbed* into i). The minimum degree algorithm selects a supernode k of least degree and eliminates it. All nodes start out simply representing themselves.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

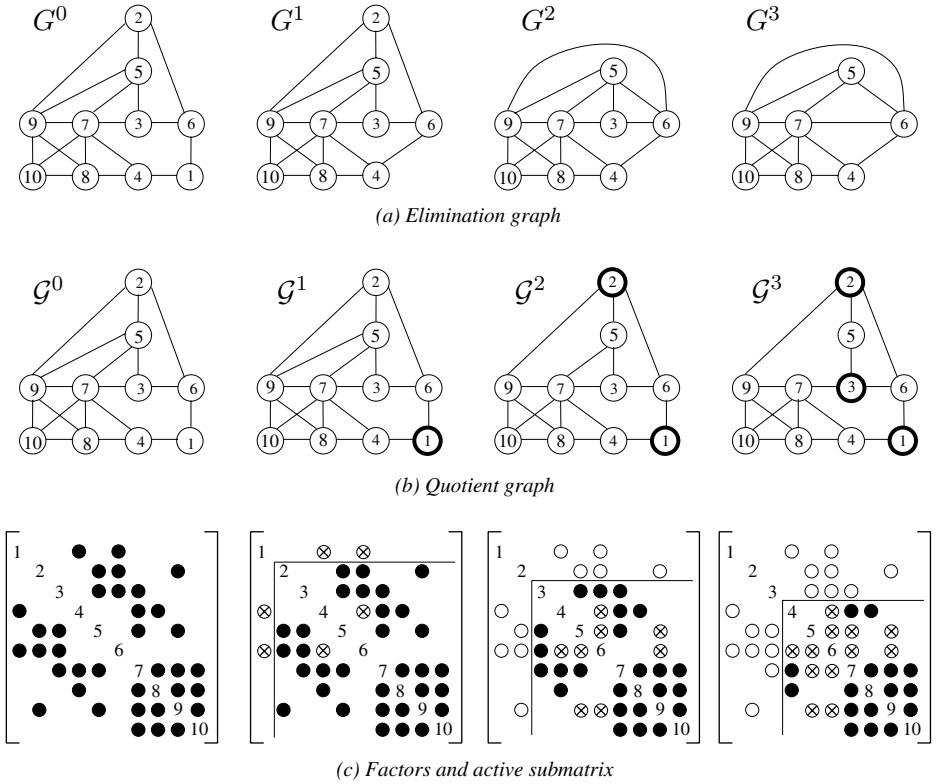


Figure 7.1. *Graph elimination*

After k is eliminated, if a node i is left with just an edge to k ($\mathcal{E}_i = \{k\}$ and $\mathcal{A}_i$ is empty), it can be eliminated immediately (called *mass elimination*). Let $|i|$ denote the number of nodes represented by supernode i. To keep the notation simple, when dealing with set expressions the use of i as a member of a set should be interpreted as the set of nodes represented by i.

Supernodes and mass elimination reduce the number of times (7.1) must be evaluated. Another technique discards the use of (7.1), and uses an approximation $\overline{d}_i$ to the true degree d_i instead, which is cheaper to compute, where

$$\overline{d}_i = |\mathcal{A}_i| + |\mathcal{L}_k \setminus \{i\}| + \sum_{e \in \mathcal{E}_i \setminus \{k\}} |\mathcal{L}_e \setminus \mathcal{L}_k| \tag{7.2}$$

and k is the current pivot element, and where $\mathcal{A}_i$ is assumed to have already been pruned. Note that $\overline{d}_i = d_i$ if $|\mathcal{E}_i| \leq 2$ (where $\mathcal{E}_i$ includes k), because $\mathcal{A}_i$ and $\mathcal{L}_k$ are disjoint after pruning. Otherwise $\overline{d}_i \geq d_i$. Using $\overline{d}$ in place of d results in the *approximate minimum degree* algorithm, or AMD. At first glance, (7.2) looks no simpler than computing the set (7.1) and then finding its size. The *scan1* algorithm

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

below shows how to compute the set differences $|\mathcal{L}_e \setminus \mathcal{L}_k|$ efficiently. In (7.2) and in the rest of the discussion, $|\mathcal{A}_i|$ and $|\mathcal{L}_e|$ refer to the sum of $|j|$ for the nodes j that they contain.

```
function scan1
    assume w(e) < 0 for all e = 1, ..., n
    for each node i ∈ L_k do
        for each element e ∈ E_i do
            if (w(e) < 0) then w(e) = |L_e|
            w(e) = w(e) − |i|
```

Then $w(e) = |\mathcal{L}_e \setminus \mathcal{L}_k|$ if $w(e) \geq 0$. If $w(e) < 0$, then the sets $\mathcal{L}_e$ and $\mathcal{L}_k$ are disjoint, and $|\mathcal{L}_e \setminus \mathcal{L}_k| = |\mathcal{L}_e|$. Once the set differences are known, a second pass over all $i \in \mathcal{L}_k$ evaluates (7.2) to compute $\overline{d}_i$. The amortized time for computing the set differences, computing $\overline{d}_i$, and pruning the set $\mathcal{A}_i$ is $O(|\mathcal{A}_i| + |\mathcal{E}_i|)$. This is much less than the $O(d_i)$ required to compute (7.1).

The minimum degree algorithm (AMD) is the most complex of the codes presented in this book. A concise version of AMD is presented below as the `cs_amd` function. It uses slightly more workspace than AMD, leading to a simpler code. It uses the tree postordering `cs_tdfs`, rather than AMD's own postordering. It has no control parameters, as AMD does, and does not compute any of the statistics that AMD does (such as $|L|$ and the floating-point operation count for a subsequent Cholesky factorization). It also has a simpler dense-node removal strategy. However, even with these simplifications, `cs_amd` generates orderings of the same quality as AMD and is just as fast.

Construct matrix `C`: The function accepts the matrix `A` as input and returns a permutation vector `p`. The `cs_amd` function operates on a symmetric matrix, so one of three symmetric matrices is formed. If `order` is `0`, a natural ordering `p=NULL` is returned. If `order` is `1` and the matrix is square, `C=A+A'` is formed, which is appropriate for a Cholesky factorization or an LU factorization of a matrix with substantial entries on the diagonal and a roughly symmetric nonzero pattern (using `tol<1` for `cs_lu`). If `order` is `2`, `C=A'*A` is formed after removing "dense" rows from `A`. This is suitable for LU factorization of unsymmetric matrices and is similar to what COLAMD computes. If `order` is `3`, `C=A'*A` is computed, which is best used for QR factorization or for LU factorization if `A` has no dense rows. A "dense" row is a row with more than $10\sqrt{n}$ entries.

Diagonal entries are removed from `C` (since `cs_amd` requires a graph with no self-edges), and extra "elbow room" is added to `C->i` via `cs_sprealloc`. The contents of `C` will be destroyed during the elimination (it holds $\mathcal{G}^{[k]}$). After `C` is formed, the output `p` and workspace of size $8(n+1)$ are allocated. The input `A` is not modified. To simplify the remainder of the discussion, the superscript $[k]$ is dropped.

Initialize quotient graph: The quotient graph $\mathcal{G}$ is represented by the arrays `Cp`, `Ci`, `w`, `nv`, `elen`, and `len`, and `degree`, each of size `n+1`, except for `Ci` of size `nzmax`. There are four kinds of nodes and elements that must be represented:

- A *live* node is a node `i` (or a supernode) that has not been selected as a

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

pivot and has not been merged into another supernode. In this case, $\mathcal{E}_i$ is represented as `Ci [Cp[i] ... Cp[i]+elen[i]-1]`, where `elen[i]` ≥ 0. The set $\mathcal{A}_i$ is represented as `Ci [Cp[i]+elen[i] ... Cp[i]+len[i]-1]`. Note that `Cp[i]` is greater than or equal to zero. The number of original nodes represented by `i` is given by $|i| =$ `nv[i]`, which is thus greater than zero. The degree $\overline{d}_i$ is `degree[i]`.

- A *dead* node `i` is one that has been removed from the graph, having been absorbed into node `r = CS_FLIP(Cp[i])`, where `CS_FLIP(x)` is defined as `-(x)-2`. Note that `Cp[i]` is less than zero. Note that node `r` might itself be absorbed into yet another node. In this case `Cp` forms an *assembly tree*, very similar to the elimination tree. The adjacency list of i is not stored. `elen[i]` is set to `-1` to denote the fact that node `i` is dead. The size of node `i`, $|i| =$ `nv[i]`, is zero.

- A live element `e` is one that is in the graph $\mathcal{G}$, having been formed when node `e` was selected as the pivot. `elen[e]` is set to `-2`, and `w[e]` will always be greater than zero. The sets $\mathcal{A}_e$ and $\mathcal{E}_e$ do not exist. Instead, the set $\mathcal{L}_e$ is stored in `Ci [Cp[e] ... Cp[e]+len[e]-1]`. `degree[e]` is $|\mathcal{L}_e|$, which is not the same as `len[e]`; the latter is smaller because $\mathcal{L}_e$ is a list of supernodes. The size of node `e`, $|e| =$ `nv[e]`, is greater than zero. It represents the number of nodes represented by supernode `e` when it was selected as the pivot.

- A dead element `e` is one that as been absorbed into a subsequent element `s = CS_FLIP(Cp[e])`. `elen[e]` is `-2` and `w[e]` is set to zero to denote that `e` is a dead element. $|e| =$ `nv[e]` > 0 is the same as for live elements.

`cs_amd` initializes the quotient graph $\mathcal{G}$ and two sets of `n` linked lists: the *degree lists* and the *hash buckets*. Degree list `d` is a doubly linked list containing a list of all nodes with approximate degree `d`. The head of list `d` is `head[d]`. The nodes preceding and following node `i` in the list are `last[i]` and `next[i]`, respectively. The hash buckets share some of this workspace; `hhead[h]` is the head of the `h`th hash bucket, a singly linked list. Since a node is never in both lists, `next[i]` is the node following `i` in the list, and `last[i]` is the hash key for node `i`. The degree lists are used to determine the node of minimum degree, and the hash buckets are used for supernode detection.

Initialize degree lists: Each node is placed in its degree lists. Nodes of zero degree are eliminated immediately. Nodes with degree $\geq$ `dense` are also eliminated and merged into a placeholder node `n`, a dead element. These dense nodes will appear last in the output permutation `p`.

Select node of minimum approximate degree: `cs_amd` is now ready to start eliminating the graph. It first finds a node `k` of minimum degree and removes it from its degree list. The variable `nel` keeps track of how many nodes have been eliminated; the elimination of `k` increments this count by $|k| =$ `nv[k]`. Because nodes are not eliminated in order `0` through `n-1`, this pivot node `k` is not equivalent to the `k` discussed above, but it serves the same purpose.

Garbage collection: The new element $\mathcal{L}_k$ requires space in `Ci`. It will be placed at the end of this array, in `Ci[cnz ... cnz +` $|\mathcal{L}_k| - 1$`]`, if $|\mathcal{E}_k| > 0$ (more

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

precisely, less space than this will be used; the exact degree $d_k = |\mathcal{L}_k|$ is an upper bound, and $\overline{d}_k \geq d_k$ is yet a higher upper bound). If not enough space is available, garbage collection is performed to pack $\mathcal{G}$ in the `Ci` array.

Live nodes and elements need not appear in any particular order in `Ci`. To pack `Ci` efficiently, the method relies on the fact that all entries in `Ci[0...cnz-1]` are nonnegative, and `Cp` will be redefined for live nodes and elements. The `for j` loop copies the first entry from each live node and element `j` into `Cp[j]` and places `CS_FLIP(j)` in the first position of each live object. The second loop scans all of `Ci`, looking for negative entries. When a negative entry is found, the live node or element `j` is compacted. Garbage collection occurs rarely.

Construct new element: The new element $\mathcal{L}_k$ is constructed, using (7.1). It is constructed in place if $|\mathcal{E}_k| = 0$. `nv[i]` is negated for all nodes `i` $\in \mathcal{L}_k$ to flag them as members of this set. Each node `i` is removed from the degree lists. All elements `e` $\in \mathcal{E}_k$ are absorbed into element k.

Find set differences: The *scan1* function now computes the set differences $|\mathcal{L}_e \setminus \mathcal{L}_k|$ for all elements e. At the start of the scan, no entry in the `w` array is greater than or equal to `mark`. A test is made to ensure `mark` + $\max |\mathcal{L}_e|$ does not cause integer overflow. If it does, then `w` is safely reset and the algorithm continues. The value of `mark` is used as an offset; $w(e)$ in the *scan1* pseudocode is replaced with `w[e]-mark` in the code.

Degree update: The second pass (*scan2*) computes the approximate degree $\overline{d}_i$ using (7.2), prunes the sets $\mathcal{E}_i$ and $\mathcal{A}_i$, and computes a hash function $h(i)$ for all nodes in $\mathcal{L}_k$. The hash function will be used in the next step for supernode detection. If a live element e is found where $|\mathcal{L}_e \setminus \mathcal{L}_k| = 0$, then *aggressive* element absorption is performed. Element e is a subset of k, so it is not needed to represent $\mathcal{G}$. At this point, `degree[i]` = `d` = $\overline{d}_i - |\mathcal{L}_k \setminus \{i\}|$ is computed for node `i` (7.2). The $|\mathcal{L}_k \setminus \{i\}|$ term is added later, after mass elimination and supernode detection. If `d` is zero, node `i` is mass eliminated along with element `k`; otherwise, node `i` remains alive. Element `k` is added to $\mathcal{E}_i$, and node `i` is placed in the `h`th hash bucket. Finally, `mark` is incremented by $\max |\mathcal{L}_e|$ to ensure that all entries in `w` are less than `mark`.

Supernode detection: Supernode detection relies on the hash function $h(i)$ computed for each node `i`. If two nodes have identical adjacency lists, their hash functions will be identical. Each hash bucket containing any node `i` $\in \mathcal{L}_k$ is considered. The first node `i` in the hash bucket is compared with all other nodes `j`; this is repeated until the hash bucket is empty. To compare `i` with a set of other nodes `j`, `w[s]=mark` is set for each node or element `s` in $\mathcal{A}_i$ or $\mathcal{E}_i$. These lists have been pruned in *scan2* of all dead nodes and elements. If the adjacent lists of `i` and `j` have the same lengths and all `s` in $\mathcal{A}_j$ or $\mathcal{E}_j$ are flagged, then `i` and `j` are identical. In this case, `j` is absorbed into `i` and removed from the hash bucket. The `mark` is incremented to clear the array `w` for the next iteration.

Finalize new element: The elimination of node `k` is nearly complete. All nodes `i` in $\mathcal{L}_k$ are scanned one last time. Node `i` is removed from $\mathcal{L}_k$ if it is dead (it may have been absorbed during supernode detection). The flagged status of `nv[i]` is cleared. The degree $\overline{d}_i$ is finalized, and node i is placed in its corresponding degree list. The new minimum degree is found when nodes are placed back into the degree lists. Note that the degree of the current element, `dk` = $|\mathcal{L}_k \setminus \{i\}|$, is finally added to

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

the degree of each node during this final pass to complete the approximate degree computation for (7.2). This term was not added in *scan2*, because it was modified during that scan due to mass elimination. Finally, `nv[k]` is updated to reflect the number of nodes represented by `k` (this may have increased, since `k` was selected as the pivot, due to mass elimination). If the set $\mathcal{L}_k$ is empty, element `k` is a root of the assembly tree, and element `k` is removed from the graph.

Postordering: The elimination is complete, but no permutation has been computed. All that is left of the graph is the assembly tree (`Cp`) and a set of dead nodes and elements (`i` is a dead node if `nv[i]` is zero and a dead element if `nv[i]` > 0). It is from this information only that the final permutation is computed. The tree is restored by unflipping all of `Cp`. It now forms a tree; `Cp[x]` is the parent of `x` or `-1` if `x` is a root. This is not the elimination tree, but it is quite similar.

If an element `e` has been absorbed into its parent `Cp[e]`, then `e` must precede `Cp[e]` in the output permutation `p`. Likewise, a node `i` must precede its parent `Cp[i]`. A distinction must be made between nodes and elements. The parent of an element is always an element. The parent of a node can be either another node or an element, but a node can never be a root of the tree. The children of an element `e` must appear before it in `p`, but all child elements must appear before all child nodes, because child nodes (and their descendants in the assembly tree) reflect a set of nodes that were absorbed into supernode `e` when it was selected as a pivot.

A postordering of the assembly tree gives the permutation `p`. The list of children of any node `x` are partitioned; the child elements appear first, followed by the child nodes. The dead element `n` is a placeholder for any dense rows and columns of `C`, so it too is included in the postordering; it and its descendants will be ordered at the very last, followed by `n=p[n]` itself. Thus, `p[0...n-1]` is the resulting fill-reducing permutation. This postordering is much simpler than the postordering in AMD, yet just as effective.

```
int *cs_amd (int order, const cs *A)  /* order 0:natural, 1:Chol, 2:LU, 3:QR */
{
    cs *C, *A2, *AT ;
    int *Cp, *Ci, *last, *W, *len, *nv, *next, *P, *head, *elen, *degree, *w,
        *hhead, *ATp, *ATi, d, dk, dext, lemax = 0, e, elenk, eln, i, j, k, k1,
        k2, k3, jlast, ln, dense, nzmax, mindeg = 0, nvi, nvj, nvk, mark, wnvi,
        ok, cnz, nel = 0, p, p1, p2, p3, p4, pj, pk, pk1, pk2, pn, q, n, m, t ;
    unsigned int h ;
    /* --- Construct matrix C ----------------------------------------------- */
    if (!CS_CSC (A) || order <= 0 || order > 3) return (NULL) ; /* check */
    AT = cs_transpose (A, 0) ;              /* compute A' */
    if (!AT) return (NULL) ;
    m = A->m ; n = A->n ;
    dense = CS_MAX (16, 10 * sqrt ((double) n)) ;   /* find dense threshold */
    dense = CS_MIN (n-2, dense) ;
    if (order == 1 && n == m)
    {
        C = cs_add (A, AT, 0, 0) ;          /* C = A+A' */
    }
    else if (order == 2)
    {
        ATp = AT->p ;                       /* drop dense columns from AT */
        ATi = AT->i ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    for (p2 = 0, j = 0 ; j < m ; j++)
    {
        p = ATp [j] ;                   /* column j of AT starts here */
        ATp [j] = p2 ;                  /* new column j starts here */
        if (ATp [j+1] - p > dense) continue ;   /* skip dense col j */
        for ( ; p < ATp [j+1] ; p++) ATi [p2++] = ATi [p] ;
    }
    ATp [m] = p2 ;                      /* finalize AT */
    A2 = cs_transpose (AT, 0) ;         /* A2 = AT' */
    C = A2 ? cs_multiply (AT, A2) : NULL ;  /* C=A'*A with no dense rows */
    cs_spfree (A2) ;
}
else
{
    C = cs_multiply (AT, A) ;           /* C=A'*A */
}
cs_spfree (AT) ;
if (!C) return (NULL) ;
cs_fkeep (C, &cs_diag, NULL) ;          /* drop diagonal entries */
Cp = C->p ;
cnz = Cp [n] ;
P = cs_malloc (n+1, sizeof (int)) ;     /* allocate result */
W = cs_malloc (8*(n+1), sizeof (int)) ; /* get workspace */
t = cnz + cnz/5 + 2*n ;                 /* add elbow room to C */
if (!P || !W || !cs_sprealloc (C, t)) return (cs_idone (P, C, W, 0)) ;
len  = W           ; nv     = W +   (n+1) ; next   = W + 2*(n+1) ;
head = W + 3*(n+1) ; elen   = W + 4*(n+1) ; degree = W + 5*(n+1) ;
w    = W + 6*(n+1) ; hhead  = W + 7*(n+1) ;
last = P ;                              /* use P as workspace for last */
/* --- Initialize quotient graph ---------------------------------------- */
for (k = 0 ; k < n ; k++) len [k] = Cp [k+1] - Cp [k] ;
len [n] = 0 ;
nzmax = C->nzmax ;
Ci = C->i ;
for (i = 0 ; i <= n ; i++)
{
    head [i] = -1 ;                     /* degree list i is empty */
    last [i] = -1 ;
    next [i] = -1 ;
    hhead [i] = -1 ;                    /* hash list i is empty */
    nv [i] = 1 ;                        /* node i is just one node */
    w [i] = 1 ;                         /* node i is alive */
    elen [i] = 0 ;                      /* Ek of node i is empty */
    degree [i] = len [i] ;              /* degree of node i */
}
mark = cs_wclear (0, 0, w, n) ;         /* clear w */
elen [n] = -2 ;                         /* n is a dead element */
Cp [n] = -1 ;                           /* n is a root of assembly tree */
w [n] = 0 ;                             /* n is a dead element */
/* --- Initialize degree lists ------------------------------------------ */
for (i = 0 ; i < n ; i++)
{
    d = degree [i] ;
    if (d == 0)                         /* node i is empty */
    {
        elen [i] = -2 ;                 /* element i is dead */
        nel++ ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
            Cp [i] = -1 ;                   /* i is a root of assembly tree */
            w [i] = 0 ;
        }
        else if (d > dense)                 /* node i is dense */
        {
            nv [i] = 0 ;                    /* absorb i into element n */
            elen [i] = -1 ;                 /* node i is dead */
            nel++ ;
            Cp [i] = CS_FLIP (n) ;
            nv [n]++ ;
        }
        else
        {
            if (head [d] != -1) last [head [d]] = i ;
            next [i] = head [d] ;           /* put node i in degree list d */
            head [d] = i ;
        }
    }
    while (nel < n)                         /* while (selecting pivots) do */
    {
        /* --- Select node of minimum approximate degree -------------------- */
        for (k = -1 ; mindeg < n && (k = head [mindeg]) == -1 ; mindeg++) ;
        if (next [k] != -1) last [next [k]] = -1 ;
        head [mindeg] = next [k] ;          /* remove k from degree list */
        elenk = elen [k] ;                  /* elenk = |Ek| */
        nvk = nv [k] ;                      /* # of nodes k represents */
        nel += nvk ;                        /* nv[k] nodes of A eliminated */
        /* --- Garbage collection ------------------------------------------- */
        if (elenk > 0 && cnz + mindeg >= nzmax)
        {
            for (j = 0 ; j < n ; j++)
            {
                if ((p = Cp [j]) >= 0)      /* j is a live node or element */
                {
                    Cp [j] = Ci [p] ;       /* save first entry of object */
                    Ci [p] = CS_FLIP (j) ;  /* first entry is now CS_FLIP(j) */
                }
            }
            for (q = 0, p = 0 ; p < cnz ; ) /* scan all of memory */
            {
                if ((j = CS_FLIP (Ci [p++])) >= 0)  /* found object j */
                {
                    Ci [q] = Cp [j] ;       /* restore first entry of object */
                    Cp [j] = q++ ;          /* new pointer to object j */
                    for (k3 = 0 ; k3 < len [j]-1 ; k3++) Ci [q++] = Ci [p++] ;
                }
            }
            cnz = q ;                       /* Ci [cnz...nzmax-1] now free */
        }
        /* --- Construct new element ---------------------------------------- */
        dk = 0 ;
        nv [k] = -nvk ;                     /* flag k as in Lk */
        p = Cp [k] ;
        pk1 = (elenk == 0) ? p : cnz ;      /* do in place if elen[k] == 0 */
        pk2 = pk1 ;
        for (k1 = 1 ; k1 <= elenk + 1 ; k1++)
        {
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
        if (k1 > elenk)
        {
            e = k ;                     /* search the nodes in k */
            pj = p ;                    /* list of nodes starts at Ci[pj]*/
            ln = len [k] - elenk ;      /* length of list of nodes in k */
        }
        else
        {
            e = Ci [p++] ;              /* search the nodes in e */
            pj = Cp [e] ;
            ln = len [e] ;              /* length of list of nodes in e */
        }
        for (k2 = 1 ; k2 <= ln ; k2++)
        {
            i = Ci [pj++] ;
            if ((nvi = nv [i]) <= 0) continue ; /* node i dead, or seen */
            dk += nvi ;                 /* degree[Lk] += size of node i */
            nv [i] = -nvi ;             /* negate nv[i] to denote i in Lk*/
            Ci [pk2++] = i ;            /* place i in Lk */
            if (next [i] != -1) last [next [i]] = last [i] ;
            if (last [i] != -1)         /* remove i from degree list */
            {
                next [last [i]] = next [i] ;
            }
            else
            {
                head [degree [i]] = next [i] ;
            }
        }
        if (e != k)
        {
            Cp [e] = CS_FLIP (k) ;      /* absorb e into k */
            w [e] = 0 ;                 /* e is now a dead element */
        }
    }
    if (elenk != 0) cnz = pk2 ;         /* Ci [cnz...nzmax] is free */
    degree [k] = dk ;                   /* external degree of k - |Lk\i| */
    Cp [k] = pk1 ;                      /* element k is in Ci[pk1..pk2-1] */
    len [k] = pk2 - pk1 ;
    elen [k] = -2 ;                     /* k is now an element */
    /* --- Find set differences ----------------------------------------- */
    mark = cs_wclear (mark, lemax, w, n) ;  /* clear w if necessary */
    for (pk = pk1 ; pk < pk2 ; pk++)    /* scan 1: find |Le\Lk| */
    {
        i = Ci [pk] ;
        if ((eln = elen [i]) <= 0) continue ;/* skip if elen[i] empty */
        nvi = -nv [i] ;                     /* nv [i] was negated */
        wnvi = mark - nvi ;
        for (p = Cp [i] ; p <= Cp [i] + eln - 1 ; p++)  /* scan Ei */
        {
            e = Ci [p] ;
            if (w [e] >= mark)
            {
                w [e] -= nvi ;          /* decrement |Le\Lk| */
            }
            else if (w [e] != 0)        /* ensure e is a live element */
            {
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
            w [e] = degree [e] + wnvi ; /* 1st time e seen in scan 1 */
        }
    }
}
/* --- Degree update ------------------------------------------------ */
for (pk = pk1 ; pk < pk2 ; pk++)    /* scan2: degree update */
{
    i = Ci [pk] ;                   /* consider node i in Lk */
    p1 = Cp [i] ;
    p2 = p1 + elen [i] - 1 ;
    pn = p1 ;
    for (h = 0, d = 0, p = p1 ; p <= p2 ; p++)    /* scan Ei */
    {
        e = Ci [p] ;
        if (w [e] != 0)             /* e is an unabsorbed element */
        {
            dext = w [e] - mark ;   /* dext = |Le\Lk| */
            if (dext > 0)
            {
                d += dext ;         /* sum up the set differences */
                Ci [pn++] = e ;     /* keep e in Ei */
                h += e ;            /* compute the hash of node i */
            }
            else
            {
                Cp [e] = CS_FLIP (k) ;  /* aggressive absorb. e->k */
                w [e] = 0 ;             /* e is a dead element */
            }
        }
    }
    elen [i] = pn - p1 + 1 ;        /* elen[i] = |Ei| */
    p3 = pn ;
    p4 = p1 + len [i] ;
    for (p = p2 + 1 ; p < p4 ; p++) /* prune edges in Ai */
    {
        j = Ci [p] ;
        if ((nvj = nv [j]) <= 0) continue ; /* node j dead or in Lk */
        d += nvj ;                  /* degree(i) += |j| */
        Ci [pn++] = j ;             /* place j in node list of i */
        h += j ;                    /* compute hash for node i */
    }
    if (d == 0)                     /* check for mass elimination */
    {
        Cp [i] = CS_FLIP (k) ;      /* absorb i into k */
        nvi = -nv [i] ;
        dk -= nvi ;                 /* |Lk| -= |i| */
        nvk += nvi ;                /* |k| += nv[i] */
        nel += nvi ;
        nv [i] = 0 ;
        elen [i] = -1 ;             /* node i is dead */
    }
    else
    {
        degree [i] = CS_MIN (degree [i], d) ;   /* update degree(i) */
        Ci [pn] = Ci [p3] ;         /* move first node to end */
        Ci [p3] = Ci [p1] ;         /* move 1st el. to end of Ei */
        Ci [p1] = k ;               /* add k as 1st element in of Ei */
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
            len [i] = pn - p1 + 1 ;     /* new len of adj. list of node i */
            h %= n ;                    /* finalize hash of i */
            next [i] = hhead [h] ;      /* place i in hash bucket */
            hhead [h] = i ;
            last [i] = h ;              /* save hash of i in last[i] */
        }
    }                                   /* scan2 is done */
    degree [k] = dk ;                   /* finalize |Lk| */
    lemax = CS_MAX (lemax, dk) ;
    mark = cs_wclear (mark+lemax, lemax, w, n) ;    /* clear w */
    /* --- Supernode detection ------------------------------------------ */
    for (pk = pk1 ; pk < pk2 ; pk++)
    {
        i = Ci [pk] ;
        if (nv [i] >= 0) continue ;         /* skip if i is dead */
        h = last [i] ;                      /* scan hash bucket of node i */
        i = hhead [h] ;
        hhead [h] = -1 ;                    /* hash bucket will be empty */
        for ( ; i != -1 && next [i] != -1 ; i = next [i], mark++)
        {
            ln = len [i] ;
            eln = elen [i] ;
            for (p = Cp [i]+1 ; p <= Cp [i] + ln-1 ; p++) w [Ci [p]] = mark;
            jlast = i ;
            for (j = next [i] ; j != -1 ; ) /* compare i with all j */
            {
                ok = (len [j] == ln) && (elen [j] == eln) ;
                for (p = Cp [j] + 1 ; ok && p <= Cp [j] + ln - 1 ; p++)
                {
                    if (w [Ci [p]] != mark) ok = 0 ;    /* compare i and j*/
                }
                if (ok)                     /* i and j are identical */
                {
                    Cp [j] = CS_FLIP (i) ;  /* absorb j into i */
                    nv [i] += nv [j] ;
                    nv [j] = 0 ;
                    elen [j] = -1 ;         /* node j is dead */
                    j = next [j] ;          /* delete j from hash bucket */
                    next [jlast] = j ;
                }
                else
                {
                    jlast = j ;             /* j and i are different */
                    j = next [j] ;
                }
            }
        }
    }
    /* --- Finalize new element------------------------------------------ */
    for (p = pk1, pk = pk1 ; pk < pk2 ; pk++)   /* finalize Lk */
    {
        i = Ci [pk] ;
        if ((nvi = -nv [i]) <= 0) continue ;/* skip if i is dead */
        nv [i] = nvi ;                      /* restore nv[i] */
        d = degree [i] + dk - nvi ;         /* compute external degree(i) */
        d = CS_MIN (d, n - nel - nvi) ;
        if (head [d] != -1) last [head [d]] = i ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
            next [i] = head [d] ;               /* put i back in degree list */
            last [i] = -1 ;
            head [d] = i ;
            mindeg = CS_MIN (mindeg, d) ;       /* find new minimum degree */
            degree [i] = d ;
            Ci [p++] = i ;                      /* place i in Lk */
        }
        nv [k] = nvk ;                      /* # nodes absorbed into k */
        if ((len [k] = p-pk1) == 0)         /* length of adj list of element k*/
        {
            Cp [k] = -1 ;                   /* k is a root of the tree */
            w [k] = 0 ;                     /* k is now a dead element */
        }
        if (elenk != 0) cnz = p ;           /* free unused space in Lk */
    }
    /* --- Postordering ----------------------------------------------------- */
    for (i = 0 ; i < n ; i++) Cp [i] = CS_FLIP (Cp [i]) ;/* fix assembly tree */
    for (j = 0 ; j <= n ; j++) head [j] = -1 ;
    for (j = n ; j >= 0 ; j--)              /* place unordered nodes in lists */
    {
        if (nv [j] > 0) continue ;          /* skip if j is an element */
        next [j] = head [Cp [j]] ;          /* place j in list of its parent */
        head [Cp [j]] = j ;
    }
    for (e = n ; e >= 0 ; e--)              /* place elements in lists */
    {
        if (nv [e] <= 0) continue ;         /* skip unless e is an element */
        if (Cp [e] != -1)
        {
            next [e] = head [Cp [e]] ;      /* place e in list of its parent */
            head [Cp [e]] = e ;
        }
    }
    for (k = 0, i = 0 ; i <= n ; i++)       /* postorder the assembly tree */
    {
        if (Cp [i] == -1) k = cs_tdfs (i, k, head, next, P, w) ;
    }
    return (cs_idone (P, C, W, 1)) ;
}
```

The `cs_wclear` function is used in `cs_amd` to clear the `w` array. The condition is true just once in the first call to `cs_wclear` and then when integer overflow is near (in which case `w` is safely reset and the algorithm continues). `cs_diag` is used to drop diagonal entries.

```
static int cs_wclear (int mark, int lemax, int *w, int n)
{
    int k ;
    if (mark < 2 || (mark + lemax < 0))
    {
        for (k = 0 ; k < n ; k++) if (w [k] != 0) w [k] = 1 ;
        mark = 2 ;
    }
    return (mark) ;     /* at this point, w [0..n-1] < mark holds */
}

static int cs_diag (int i, int j, double aij, void *other) { return (i != j) ; }
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

In MATLAB, `p=amd(A)`[12] or `p=symamd(A)` compute essentially the same ordering as `p=cs_amd(A)`. Slight differences exist due to *tie-breaking*. The input matrix `C` is computed differently in AMD and `cs_amd`; the adjacency lists are the same, but neither produce sorted adjacency lists $\mathcal{A}$. This affects the order that nodes are replaced in the degree lists, and thus the pivot order can be affected. There is often more than one node of identical minimum degree in the degree lists, and both AMD and `cs_amd` simply select the first node in the first nonempty degree list.

7.2 Maximum matching

A *maximum matching* (also called a *maximum transversal*) is a permutation of any matrix A so that its kth diagonal is zero-free and $|k|$ is uniquely minimized (except when A is completely zero). This permutation determines the structural rank of a matrix and is a precursor to LU or QR factorization or to the block triangular form and Dulmage–Mendelsohn decomposition described in the next sections. With this maximum matching, a matrix has structural full rank if and only if $k = 0$ and is structurally rank deficient otherwise. The number of entries on this diagonal gives the *structural rank* of a matrix A which is an upper bound on the numerical rank of any matrix with the same nonzero pattern as A. It is `sprank(A)` in MATLAB.

Consider the bipartite graph $G = (V, E)$ of an m-by-n matrix A with m row nodes, n column nodes, and undirected edges $E = \{(i, j) \mid a_{ij} \neq 0\}$; no edge connects pairs of row nodes or pairs of column nodes. Let $\mathcal{A}_j$ denote the nonzeros in column j or, equivalently, the rows adjacent to j in G. Note that although the edges are undirected, (i, j) and (j, i) are different edges. A *matching* is a set of rows $\mathcal{R}$ and columns $\mathcal{C}$ where each row in $i \in \mathcal{R}$ is paired with a unique $j \in \mathcal{C}$, where $(i, j) \in E$. A row $i \in \mathcal{R}$ is called a matched row, a column $j \in \mathcal{C}$ is called a matched column, and an edge (i, j) where both $i \in \mathcal{R}$ and $j \in \mathcal{C}$ is called a matched edge. All other rows, columns, and edges are unmatched. The matching defines a zero-free diagonal of a permuted submatrix (`diag(A(r,c))` in MATLAB notation). A maximum matching of G has a size greater than or equal to any other matching in G. A matching is *row-perfect* if all rows are matched and *column-perfect* if all columns are matched. A graph can have many maximum matchings (there are $n!$ maximum matchings in the graph of a square dense matrix). The size of this maximum matching is the structural rank. Fortunately, the algorithms and theorems presented here can use any maximum matching, and the matching algorithm presented below is guaranteed to find one of them.

The algorithm starts with an empty matching. Let `jmatch[i]` equal `j` if row $i \in \mathcal{R}$ is matched to column $j \in \mathcal{C}$ or `-1` if row $i \notin \mathcal{R}$. The algorithm works by extending the matching one column $j \notin \mathcal{C}$ at a time by finding an *alternating augmenting path*. The path starts at an unmatched column k and then traverses any edge to a row i_1. Since k is unmatched, this edge (k, i_1) is not matched. When reaching an unmatched row, the path stops. If i_1 is matched, the path traverses the

[12]The `amd` function is an internal part of MATLAB, but MATLAB 7.2 does not include an interface for it. It will likely be added to a future version of MATLAB. In the meantime, use `symamd` or use the version of `amd` at www.siam.org/books/fa02.

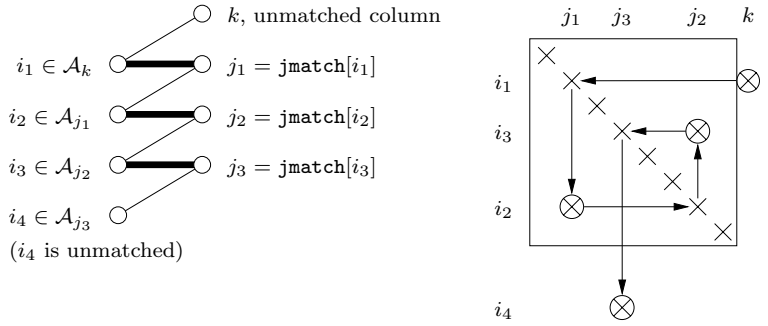


Figure 7.2. *An alternating augmenting path*

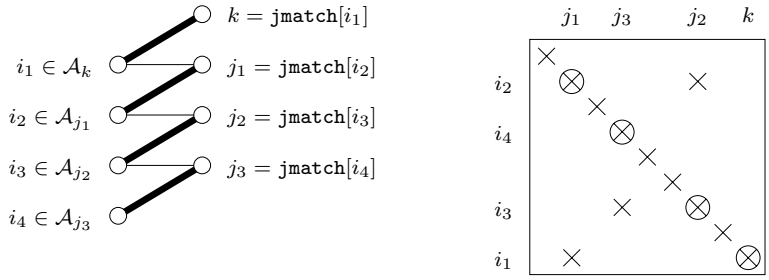


Figure 7.3. *Matching extended via an alternating augmenting path*

matched edge (i_1, j_1), where $j_1 = \texttt{jmatch}[i_1]$, and then an unmatched edge (j_1, i_2). The path continues until it stops at an unmatched row. The path is *alternating* because every other edge in the path is matched. In general the path can be of any odd length, and no node or edge appears in the path twice. An alternating augmenting path of length 7 with matched edges shown in bold,

$$(k, i_1), (\mathbf{i_1}, \mathbf{j_1}), (j_1, i_2), (\mathbf{i_2}, \mathbf{j_2}), (j_2, i_3), (\mathbf{i_3}, \mathbf{j_3}), (j_3, i_4), \tag{7.3}$$

is shown in Figure 7.2. The figure also gives a matrix view of the same path. Matched edges in the graph are shown in bold; the same edges correspond to the diagonal elements of the permuted submatrix (drawn as a box). In the matrix view, entries corresponding to unmatched edges are circled. To extend the matching (as shown in Figure 7.3), k and i_4 are added to $\mathcal{C}$ and $\mathcal{R}$, respectively, and the matching along the path is changed so that the path becomes

$$(\mathbf{k}, \mathbf{i_1}), (i_1, j_1), (\mathbf{j_1}, \mathbf{i_2}), (i_2, j_2), (\mathbf{j_2}, \mathbf{i_3}), (i_3, j_3), (\mathbf{j_3}, \mathbf{i_4}). \tag{7.4}$$

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

That is, k = `jmatch`$[i_1]$, j_1 = `jmatch`$[i_2]$, and so on. The matching has been extended by one additional edge. Note that any matched row or column remains matched; k and i_4 are added to $\mathcal{C}$ and $\mathcal{R}$, respectively, and no nodes are removed from $\mathcal{C}$ or $\mathcal{R}$. If no unmatched row i_4 is found, no such path exists and the matching is not extended (this can occur only if A is structurally rank deficient). The modified graph and matrix are shown in Figure 7.3. The four entries circled in Figure 7.2 are still circled in Figure 7.3; the rows have been permuted to place them on the diagonal, and the box denoting the current match is one row and column larger. The three formerly matched entries are no longer on the diagonal. There can of course be other unmatched edges incident on these 8 nodes, and correspondingly more off-diagonal entries in the matrix. They are left out to make the figures clearer.

The path (7.3) can be found via a depth-first search of G, starting at an unmatched column k and traversing only alternating paths. If the whole graph is searched at every step, the time complexity is $O(|A|n)$ to find the entire maximum matching for a square matrix, but typically only a small part of G needs to be traversed before finding an alternating path (at which point the search stops). To reduce the typical cost of finding a path, a one-step breadth-first search is performed at each column j before continuing in a depth-first manner (called a *cheap match*). Once a row i is matched, it remains matched (although the column `jmatch`$[i]$ it is matched to may change). The breadth-first search exploits this fact by splitting $\mathcal{A}_j$ into two parts, the first of which contains only matched rows. When considering a column j, rows in the second part of $\mathcal{A}_j$ are considered, and the splitting is extended until an unmatched row (if any) is found. Thus, any edge is considered only once in this breadth-first search, adding only $O(|A|)$ to the time for finding the entire maximum matching but greatly reducing the average-case complexity of the algorithm.

The `maxtrans` and `augment` functions are not part of CSparse, since they rely on a recursive depth-first search, which can cause stack overflow for very large matrices. However, they are simpler to understand than the nonrecursive versions, so they are discussed first. `maxtrans` allocates workspace (`w` and `cheap`) and the result `jmatch`. Initially, all rows are unmatched. `w[j]` is used to mark column node `j` during a depth-first search; `w[j]=k` if column `j` has been visited during the `k`th step of the algorithm or `w[j]<k` otherwise. The splitting of $\mathcal{A}_j$ for the one-step breadth-first search is given by `cheap[j]`; `Ai[Ap[j] ... cheap[j]-1]` is known to contain only matched rows, whereas `Ai[cheap[j] ... Ap[j+1]-1]` may contain both matched and unmatched rows.

After these initializations, the one-line `for k` loop computes the matching. It searches for an alternating augmenting path starting at column `k` and augments the matching if this path is found. At the start of the `k`th iteration, $\mathcal{C}$ is a subset of $\{0 \dots k-1\}$, and it may be extended by adding column k. When the algorithm completes, `j=jmatch[i]` $\geq$ `0` if row `i` is matched to column `j`. If row `i` is not matched, `jmatch[i]=-1`.

The recursive function `augment` is called by `maxtrans`, starting at node `j=k` (`j` will be modified when `augment` calls itself recursively, but `k` is kept unchanged). When at node `j`, it first attempts to find a cheap match (the first `for` loop). `cheap[j]` is modified to point to the remaining part of $\mathcal{A}_j$. If no cheap match

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

is found, all edges $i \in \mathcal{A}_j$ are considered in a depth-first manner. All of these rows `i` must be already matched; otherwise, a cheap match would have already been found and this loop would be skipped. If column `jmatch[i]` has not been considered during this `k`th step, a recursive call is made to find an augmenting path starting at column `jmatch[i]` (corresponding, for example, to column j_1 in Figure 7.2). The loop is terminated if a path is found, and the matching is revised by matching this new row `i` to column `j`.

```
int *maxtrans (cs *A)   /* returns jmatch [0..m-1] */
{
    int i, j, k, n, m, *Ap, *jmatch, *w, *cheap ;
    if (!A) return (NULL) ;                          /* check inputs */
    n = A->n ; m = A->m ; Ap = A->p ;
    jmatch = cs_malloc (m, sizeof (int)) ;           /* allocate result */
    w = cs_malloc (2*n, sizeof (int)) ;              /* allocate workspace */
    if (!w || !jmatch) return (cs_idone (jmatch, NULL, w, 0)) ;
    cheap = w + n ;
    for (j = 0 ; j < n ; j++) cheap [j] = Ap [j] ;  /* for cheap assignment */
    for (j = 0 ; j < n ; j++) w [j] = -1 ;           /* all columns unflagged */
    for (i = 0 ; i < m ; i++) jmatch [i] = -1 ;      /* no rows matched yet */
    for (k = 0 ; k < n ; k++) augment (k, A, jmatch, cheap, w, k) ;
    return (cs_idone (jmatch, NULL, w, 1)) ;
}

int augment (int k, cs *A, int *jmatch, int *cheap, int *w, int j)
{
    int found = 0, p, i = -1, *Ap = A->p, *Ai = A->i ;
    /* --- Start depth-first-search at node j ------------------------------- */
    w [j] = k ;                                      /* mark j as visited for kth path */
    for (p = cheap [j] ; p < Ap [j+1] && !found ; p++)
    {
        i = Ai [p] ;                                 /* try a cheap assignment (i,j) */
        found = (jmatch [i] == -1) ;
    }
    cheap [j] = p ;                                  /* start here next time for j */
    /* --- Depth-first-search of neighbors of j ----------------------------- */
    for (p = Ap [j] ; p < Ap [j+1] && !found ; p++)
    {
        i = Ai [p] ;                                 /* consider row i */
        if (w [jmatch [i]] == k) continue ; /* skip col jmatch [i] if marked */
        found = augment (k, A, jmatch, cheap, w, jmatch [i]) ;
    }
    if (found) jmatch [i] = j ;                      /* augment jmatch if path found */
    return (found) ;
}
```

Just as with the recursive depth-first search algorithm `dfsr` described in Section 3.2, using recursion limits `augment` to solving problems of modest size. The nonrecursive `cs_maxtrans` below is essentially the same as `maxtrans`; it simply allocates more workspace and calls the nonrecursive `cs_augment` instead. The extra workspace is a stack that holds the intermediate values of the variables `j`, `i`, and `p` in the recursive `augment`. The algorithm is more efficient than `maxtrans` for a matrix A with more rows than columns (it transposes the matrix in this case). It also returns quickly if the diagonal is already zero-free.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

The `cs_augment` function does the same thing as `augment(k, ..., k)`. First, the node `j` is placed on the stack `js`. The `while` loop continues until either the stack is empty or the last node in an augmenting path is found.

```
int *cs_maxtrans (const cs *A, int seed)  /*[jmatch [0..m-1]; imatch [0..n-1]]*/
{
    int i, j, k, n, m, p, n2 = 0, m2 = 0, *Ap, *jimatch, *w, *cheap, *js, *is,
        *ps, *Ai, *Cp, *jmatch, *imatch, *q ;
    cs *C ;
    if (!CS_CSC (A)) return (NULL) ;                /* check inputs */
    n = A->n ; m = A->m ; Ap = A->p ; Ai = A->i ;
    w = jimatch = cs_calloc (m+n, sizeof (int)) ;   /* allocate result */
    if (!jimatch) return (NULL) ;
    for (k = 0, j = 0 ; j < n ; j++)    /* count nonempty rows and columns */
    {
        n2 += (Ap [j] < Ap [j+1]) ;
        for (p = Ap [j] ; p < Ap [j+1] ; p++)
        {
            w [Ai [p]] = 1 ;
            k += (j == Ai [p]) ;        /* count entries already on diagonal */
        }
    }
    if (k == CS_MIN (m,n))              /* quick return if diagonal zero-free */
    {
        jmatch = jimatch ; imatch = jimatch + m ;
        for (i = 0 ; i < k ; i++) jmatch [i] = i ;
        for (      ; i < m ; i++) jmatch [i] = -1 ;
        for (j = 0 ; j < k ; j++) imatch [j] = j ;
        for (      ; j < n ; j++) imatch [j] = -1 ;
        return (cs_idone (jimatch, NULL, NULL, 1)) ;
    }
    for (i = 0 ; i < m ; i++) m2 += w [i] ;
    C = (m2 < n2) ? cs_transpose (A,0) : ((cs *) A) ; /* transpose if needed */
    if (!C) return (cs_idone (jimatch, (m2 < n2) ? C : NULL, NULL, 0)) ;
    n = C->n ; m = C->m ; Cp = C->p ;
    jmatch = (m2 < n2) ? jimatch + n : jimatch ;
    imatch = (m2 < n2) ? jimatch : jimatch + m ;
    w = cs_malloc (5*n, sizeof (int)) ;             /* get workspace */
    if (!w) return (cs_idone (jimatch, (m2 < n2) ? C : NULL, w, 0)) ;
    cheap = w + n ; js = w + 2*n ; is = w + 3*n ; ps = w + 4*n ;
    for (j = 0 ; j < n ; j++) cheap [j] = Cp [j] ;  /* for cheap assignment */
    for (j = 0 ; j < n ; j++) w [j] = -1 ;          /* all columns unflagged */
    for (i = 0 ; i < m ; i++) jmatch [i] = -1 ;     /* nothing matched yet */
    q = cs_randperm (n, seed) ;                     /* q = random permutation */
    for (k = 0 ; k < n ; k++)   /* augment, starting at column q[k] */
    {
        cs_augment (q ? q [k]: k, C, jmatch, cheap, w, js, is, ps) ;
    }
    cs_free (q) ;
    for (j = 0 ; j < n ; j++) imatch [j] = -1 ;     /* find row match */
    for (i = 0 ; i < m ; i++) if (jmatch [i] >= 0) imatch [jmatch [i]] = i ;
    return (cs_idone (jimatch, (m2 < n2) ? C : NULL, w, 1)) ;
}
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
static void cs_augment (int k, const cs *A, int *jmatch, int *cheap, int *w,
        int *js, int *is, int *ps)
{
    int found = 0, p, i = -1, *Ap = A->p, *Ai = A->i, head = 0, j ;
    js [0] = k ;                        /* start with just node k in jstack */
    while (head >= 0)
    {
        /* --- Start (or continue) depth-first-search at node j ------------- */
        j = js [head] ;                 /* get j from top of jstack */
        if (w [j] != k)                 /* 1st time j visited for kth path */
        {
            w [j] = k ;                 /* mark j as visited for kth path */
            for (p = cheap [j] ; p < Ap [j+1] && !found ; p++)
            {
                i = Ai [p] ;            /* try a cheap assignment (i,j) */
                found = (jmatch [i] == -1) ;
            }
            cheap [j] = p ;             /* start here next time j is traversed*/
            if (found)
            {
                is [head] = i ;         /* column j matched with row i */
                break ;                 /* end of augmenting path */
            }
            ps [head] = Ap [j] ;        /* no cheap match: start dfs for j */
        }
        /* --- Depth-first-search of neighbors of j ------------------------- */
        for (p = ps [head] ; p < Ap [j+1] ; p++)
        {
            i = Ai [p] ;                /* consider row i */
            if (w [jmatch [i]] == k) continue ; /* skip jmatch [i] if marked */
            ps [head] = p + 1 ;         /* pause dfs of node j */
            is [head] = i ;             /* i will be matched with j if found */
            js [++head] = jmatch [i] ;  /* start dfs at column jmatch [i] */
            break ;
        }
        if (p == Ap [j+1]) head-- ;     /* node j is done; pop from stack */
    }                                   /* augment the match if path found: */
    if (found) for (p = head ; p >= 0 ; p--) jmatch [is [p]] = js [p] ;
}
```

Start (or continue) depth-first search of node `j`: At each iteration of the `while` loop in `cs_augment`, the node `j` at the top of the stack is considered. If this is the first time `j` is considered, `w[j]` will not be equal to `k`; this corresponds to the start of a call to `augment(k,...,j)`. Thus, the next seven lines of code (from `w[j]=k` to `cheap[j]=p`) are identical to the same seven lines in `augment`. If a cheap match is found, this is recorded in the stack `is`, and the "recursion" will start to unwind (the `while` loop terminates). Otherwise, the starting position `p=Ap[j]` in $\mathcal{A}_j$ for the depth-first part is saved in `ps`, corresponding to the initialization of the `for (p=Ap[j] ...)` loop in `augment`.

Depth-first search of neighbors of `j`: The `for (p=ps[head] ...)` loop starts (or continues) the corresponding `for (p=Ap[j] ...)` loop in `augment`. If an unmarked column `jmatch[i]` is found, the iteration for node `j` is paused, and the new column `jmatch[i]` is now at the top of the stack. The `break` statement terminates the `for` loop so that the outermost `while` loop can consider this node

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

`jmatch[i]` at the top of the stack. If the `for` loop terminates after searching all rows $i \in \mathcal{A}_j$, then no match is found (yet), and `j` is popped from the stack by decrementing `head`. Finally, if an augmenting path is found the "recursion" unwinds by revising the matching for all unmatched edges (`i`, `j`) in this path, corresponding to the `jmatch[i]=j` statement in `augment`.

If a column-perfect matching is found, `imatch[0...n-1]` is a permutation of a subset of the columns of `A` (or all of the columns if `A` is square), and `A(imatch,:)` has a zero-free diagonal. The MATLAB statement `p=dmperm(A)` is identical (that is, `p` is `imatch`).

The worst-case time complexity of `cs_maxtrans` is $O(|A|n)$, but this rarely occurs in practice. `cs_maxtrans` can match the columns of A in reverse order (from `n-1` to `0`), or in a randomized order, which can help avoid this worst-case behavior. The `cs_randperm` computes a random permutation used by `cs_maxtrans`. If `seed` is zero, the identity permutation is returned (`p=NULL`). If `seed` is `-1`, the reverse permutation is returned (`p=n-1:-1:0` in MATLAB notation). Otherwise, a random permutation is returned.

```
int *cs_randperm (int n, int seed)
{
    int *p, k, j, t ;
    if (seed == 0) return (NULL) ;      /* return p = NULL (identity) */
    p = cs_malloc (n, sizeof (int)) ;   /* allocate result */
    if (!p) return (NULL) ;             /* out of memory */
    for (k = 0 ; k < n ; k++) p [k] = n-k-1 ;
    if (seed == -1) return (p) ;        /* return reverse permutation */
    srand (seed) ;                      /* get new random number seed */
    for (k = 0 ; k < n ; k++)
    {
        j = k + (rand ( ) % (n-k)) ;    /* j = rand int in range k to n-1 */
        t = p [j] ;                     /* swap p[k] and p[j] */
        p [j] = p [k] ;
        p [k] = t ;
    }
    return (p) ;
}
```

7.3 Block triangular form

The *block triangular form* and its generalization the Dulmage–Mendelsohn decomposition (described in the next section) is a useful tool for many sparse matrix algorithms and theorems. It is a permutation of a matrix A that reduces the work required for LU and QR factorization and provides a precise characterization of structurally rank-deficient matrices.

An m-by-n matrix A has the *strong Hall* [13] property if every set of k columns has nonzero entries in at least $k+1$ rows for all k in the range 1 to $n-1$. If A is square and has structural full rank but is not strong Hall, it can be permuted to

[13] An m-by-n matrix A has the *Hall* property if every set of k columns has nonzero entries in at least k rows for all k in the range 1 to n. Equivalently, A has the Hall property if and only if it has full structural column rank.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

block triangular form,

$$PAQ = \begin{bmatrix} A_{11} & \cdots & A_{1k} \\ & \ddots & \vdots \\ & & A_{kk} \end{bmatrix}, \tag{7.5}$$

where each diagonal block is square with a zero-free diagonal and has the strong Hall property. The strong Hall property implies full structural rank. The block triangular form (7.5) is unique, except that the blocks can sometimes be interchanged. There is often a choice of ordering within the blocks (the diagonal must remain zero-free). To solve $Ax = b$ with LU factorization, only the diagonal blocks need to be factorized, followed by a block backsolve for the off-diagonal blocks. No fill-in occurs in the off-diagonal blocks. Because each diagonal block is strong Hall, the theorems in Chapters 5 and 6 provide tighter bounds on the nonzero pattern of the factors.

The inverse of a strong Hall matrix has no zero entries (ignoring numerical cancellation), and thus should very rarely be computed in practice.

Permuting a square matrix with a zero-free diagonal into block triangular form is identical to finding the *strongly connected components* of a directed graph, $G(A)$. The directed graph is defined as $G(A) = (V, E)$, where $V = \{1, \ldots, n\}$ and $E = \{(i, j) \,|\, a_{ij} \neq 0\}$. That is, the nonzero pattern of A is the adjacency matrix of the directed graph $G(A)$. A strongly connected component is a maximal set of nodes such that for any pair of nodes i and j in the component, the paths $i \rightsquigarrow j$ and $j \rightsquigarrow i$ both exist in the graph.

The strongly connected components of a graph can be found in many ways. The simplest method uses two depth-first traversals, one of $G(A)$ and the second of the graph $G(A^T)$. This is simple in CSparse, because `cs_dfs` can already perform this depth-first traversal. It was presented in the context of a directed acyclic graph (the graph of L) to find the nonzero pattern $\mathcal{X} = \mathrm{Reach}_L(\mathcal{B})$ for the sparse triangular solve in Section 3.2, but nothing in the design of `cs_dfs` limits it to acyclic graphs. In general, the graph of A can have cycles, unlike the graph of L.

The first depth-first search returns a set $\mathcal{X}$ that contains all the nodes of the graph. As a set, this is not very interesting. However, the *order* in which nodes appear in $\mathcal{X}$ is very important. A node j is placed in the stack $\mathcal{X}$ in the order in which its corresponding *dfs*(j) finishes. A second depth-first traversal of $G(A^T)$, where nodes are considered in the reverse order of their finish times (from the top of the stack $\mathcal{X}$ to the bottom), determines the strongly connected components. Every new node i found in a new depth-first search in the second pass, and all nodes reachable from it in $G(A^T)$, define a unique strongly connected component of $G(A)$, denoted as $\mathcal{C}_b$. The algorithm and its implementation (`cs_scc`) are given below. The components are actually computed in reverse order; this detail is not included in the *scc* function below. See Section 7.7 for more details on why it works.

Since A is stored in compressed-column form, $\mathcal{A}_j$ is the adjacency list of node j in the graph $G(A^T)$. However, the *scc* algorithm will find a permutation that puts the adjacency matrix of the graph in block lower triangular form. A block upper triangular form is more conventional for sparse matrix computations, so `cs_scc` can be applied to the transpose. These two transposes cancel each other, so the

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

`cs_scc` algorithm is identical to the pseudocode *scc*. It finds a permutation vector `p` such that `C=A(p,p)` is in block upper triangular form (7.5) and an array `r` that determines where the blocks are in `C`; the `k`th block is `C(k1:k2,k1:k2)` in MATLAB notation, where `k1=r[k]` and `k2=r[k+1]-1`.

function *scc*(A)
  $\mathcal{X} = \text{Reach}_A(\{1 \dots n\})$
  $b = 0$
  for each node $i \in \mathcal{X}$
    if i is unmarked
      $b = b + 1$
      $\mathcal{C}_b =$*dfs*(i) of the graph $G(A^T)$

```
csd *cs_scc (cs *A)     /* matrix A temporarily modified, then restored */
{
    int n, i, k, b, nb = 0, top, *xi, *pstack, *p, *r, *Ap, *ATp, *rcopy, *Blk ;
    cs *AT ;
    csd *D ;
    if (!CS_CSC (A)) return (NULL) ;                /* check inputs */
    n = A->n ; Ap = A->p ;
    D = cs_dalloc (n, 0) ;                          /* allocate result */
    AT = cs_transpose (A, 0) ;                      /* AT = A' */
    xi = cs_malloc (2*n+1, sizeof (int)) ;          /* get workspace */
    if (!D || !AT || !xi) return (cs_ddone (D, AT, xi, 0)) ;
    Blk = xi ; rcopy = pstack = xi + n ;
    p = D->p ; r = D->r ; ATp = AT->p ;
    top = n ;
    for (i = 0 ; i < n ; i++)   /* first dfs(A) to find finish times (xi) */
    {
        if (!CS_MARKED (Ap, i)) top = cs_dfs (i, A, top, xi, pstack, NULL) ;
    }
    for (i = 0 ; i < n ; i++) CS_MARK (Ap, i) ; /* restore A; unmark all nodes*/
    top = n ;
    nb = n ;
    for (k = 0 ; k < n ; k++)   /* dfs(A') to find strongly connnected comp */
    {
        i = xi [k] ;            /* get i in reverse order of finish times */
        if (CS_MARKED (ATp, i)) continue ;  /* skip node i if already ordered */
        r [nb--] = top ;        /* node i is the start of a component in p */
        top = cs_dfs (i, AT, top, p, pstack, NULL) ;
    }
    r [nb] = 0 ;                /* first block starts at zero; shift r up */
    for (k = nb ; k <= n ; k++) r [k-nb] = r [k] ;
    D->nb = nb = n-nb ;         /* nb = # of strongly connected components */
    for (b = 0 ; b < nb ; b++)  /* sort each block in natural order */
    {
        for (k = r [b] ; k < r [b+1] ; k++) Blk [p [k]] = b ;
    }
    for (b = 0 ; b <= nb ; b++) rcopy [b] = r [b] ;
    for (i = 0 ; i < n ; i++) p [rcopy [Blk [i]]++] = i ;
    return (cs_ddone (D, AT, xi, 1)) ;
}
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

The last part of the `cs_scc` function sorts the permutation vector `p` in linear time ($O(n)$), so that the rows and columns in each block appear in their natural order. This is not essential but useful because a subsequent fill-reducing ordering algorithm can tend to give slightly better results if it is provided with the matrix in natural order, $P = I$ if the matrix consists of a single strongly connected component, and `cs_dmspy` looks prettier in MATLAB.

The `cs_dalloc`, `cs_dfree`, and `cs_ddone` functions allocate, free, and return a `csd` object that represents the strongly connected components found by `cs_scc`.

```
typedef struct cs_dmperm_results    /* cs_dmperm or cs_scc output */
{
    int *p ;        /* size m, row permutation */
    int *q ;        /* size n, column permutation */
    int *r ;        /* size nb+1, block k is rows r[k] to r[k+1]-1 in A(p,q) */
    int *s ;        /* size nb+1, block k is cols s[k] to s[k+1]-1 in A(p,q) */
    int nb ;        /* # of blocks in fine dmperm decomposition */
    int rr [5] ;    /* coarse row decomposition */
    int cc [5] ;    /* coarse column decomposition */
} csd ;


csd *cs_dalloc (int m, int n)
{
    csd *D ;
    D = cs_calloc (1, sizeof (csd)) ;
    if (!D) return (NULL) ;
    D->p = cs_malloc (m, sizeof (int)) ;
    D->r = cs_malloc (m+6, sizeof (int)) ;
    D->q = cs_malloc (n, sizeof (int)) ;
    D->s = cs_malloc (n+6, sizeof (int)) ;
    return ((!D->p || !D->r || !D->q || !D->s) ? cs_dfree (D) : D) ;
}


csd *cs_dfree (csd *D)
{
    if (!D) return (NULL) ;     /* do nothing if D already NULL */
    cs_free (D->p) ;
    cs_free (D->q) ;
    cs_free (D->r) ;
    cs_free (D->s) ;
    return (cs_free (D)) ;
}


csd *cs_ddone (csd *D, cs *C, void *w, int ok)
{
    cs_spfree (C) ;                     /* free temporary matrix */
    cs_free (w) ;                       /* free workspace */
    return (ok ? D : cs_dfree (D)) ;    /* return result if OK, else free it */
}
```

In MATLAB, `[p,q,r,s]=dmperm(A)` applied to a square matrix `A` with a zero-free diagonal will return the same block triangular form `A(p,p)` (where `p=q` and `r=s`), subject to the nonuniqueness of this form, as described above.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

7.4 Dulmage–Mendelsohn decomposition

The *Dulmage–Mendelsohn decomposition* is a generalization of (7.5). Suppose a maximum matching of A has been found (A need not be square). Let $\mathcal{R}$ be the set of matched rows and $\mathcal{C}$ be the set of matched columns. This matching partitions the rows and columns of A into four disjoint subsets:

$\overline{\mathcal{R}}$	the set of unmatched rows
$\overline{\mathcal{C}}$	the set of unmatched columns
$\mathcal{R}_1$	the set of all rows reachable from any column in $\overline{\mathcal{C}}$ via an alternating path
$\mathcal{C}_1$	the columns matched to $\mathcal{R}_1$
$\mathcal{C}_3$	the set of all columns reachable from any row in $\overline{\mathcal{R}}$ via an alternating path
$\mathcal{R}_3$	the rows matched to $\mathcal{C}_3$
$\mathcal{R}_2$	any row not in any of the above sets
$\mathcal{C}_2$	any column not in any of the above sets

Note that all rows in $\mathcal{R}_1$ are matched; if they were not, an alternating augmenting path could be found, extending the maximum matching (which is a contradiction). Likewise, all columns in $\mathcal{C}_3$ are matched. Also note that if $\mathcal{C}_1$ and $\mathcal{C}_3$ had a column in common, there would be an alternating augmenting path from $\overline{\mathcal{R}}$ to $\overline{\mathcal{C}}$ through that column. Similarly, $\mathcal{R}_1$ and $\mathcal{R}_3$ are disjoint. All rows in $\mathcal{R}_2$ are matched to some row in $\mathcal{C}_2$. Thus, $\mathcal{R}$ is divided into three disjoint subsets $\mathcal{R}_1$, $\mathcal{R}_2$, and $\mathcal{R}_3$, and $\mathcal{C}$ is divided into three disjoint subsets $\mathcal{C}_1$, $\mathcal{C}_2$, and $\mathcal{C}_3$. Given this four-way partition of the rows and columns, *any* matrix A can be permuted into the 4-by-4 block matrix

$$PAQ = \begin{array}{c} \begin{array}{cccc} \overline{\mathcal{C}} & \mathcal{C}_1 & \mathcal{C}_2 & \mathcal{C}_3 \end{array} \\ \left[\begin{array}{cccc} A_{11} & A_{12} & A_{13} & A_{14} \\ & & A_{23} & A_{24} \\ & & & A_{34} \\ & & & A_{44} \end{array}\right] \end{array} \begin{array}{c} \mathcal{R}_1 \\ \mathcal{R}_2 \\ \mathcal{R}_3 \\ \overline{\mathcal{R}} \end{array}, \tag{7.6}$$

where A_{12}, A_{23}, and A_{34} are square with a zero-free diagonal. The transpose of the matrix

$$[A_{11}A_{12}] \tag{7.7}$$

and the matrix

$$\left[\begin{array}{c} A_{34} \\ A_{44} \end{array}\right] \tag{7.8}$$

are both rectangular and have the strong Hall property. The matrix (7.7) has a perfect row-matching, and the matrix (7.8) has a perfect column-matching. If $\overline{\mathcal{C}}$ is empty, the matrix A has a column-perfect matching, and both $\mathcal{R}_1$ and $\mathcal{C}_1$ will be empty. Likewise, if $\overline{\mathcal{R}}$ is empty, the matrix A has a row-perfect matching, and both $\mathcal{R}_3$ and $\mathcal{C}_3$ will be empty. Thus, it is possible for the two matrices (7.7) and (7.8) to be empty (with no rows and columns). If they do exist, (7.7) will have more columns than rows, corresponding to the structurally underdetermined part of the system $Ax = b$, and (7.8) will have more rows than columns, corresponding to the structurally overdetermined part of the system $Ax = b$. The matrix A_{23} need not have the strong Hall property. If it does not, it can be permuted into block upper

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

triangular form, as described in Section 7.3. It has structural full rank because it is square with a zero-free diagonal. Thus, for any matrix A, LU or QR factorization can be applied to submatrices, all of which have the strong Hall property.

The permutation and partitioning of A given in (7.6) is unique, except that a different maximum matching can swap columns between $\overline{\mathcal{C}}$ and $\mathcal{C}_1$ and can swap rows between $\overline{\mathcal{R}}$ and $\mathcal{R}_3$ (but not arbitrarily; $\mathcal{C}_1$ must still be matchable to the set $\mathcal{R}_1$). Otherwise the eight sets, and their sizes, are unique. The row or column ordering within each of the eight sets is not unique in general.

The `cs_dmperm` function computes the Dulmage–Mendelsohn decomposition. It returns a `csd` object containing the row and column permutation vectors `p` and `q`. The four subsets of these permutation vectors are given by `cc` and `rr`; this determines the *coarse* decomposition, given in (7.6). The eight sets are given by

$\overline{\mathcal{C}}$	`q[0 ... cc[1]-1]`	$\mathcal{R}_1$	`p[0 ... rr[1]-1]`
$\mathcal{C}_1$	`q[cc[1] ... cc[2]-1]`	$\mathcal{R}_2$	`p[rr[1] ... rr[2]-1]`
$\mathcal{C}_2$	`q[cc[2] ... cc[3]-1]`	$\mathcal{R}_3$	`p[rr[2] ... rr[3]-1]`
$\mathcal{C}_3$	`q[cc[3] ... cc[4]-1]`	$\overline{\mathcal{R}}$	`p[rr[3] ... rr[4]-1]`

The *fine* decomposition includes the permutation of the A_{23} submatrix into its strongly connected components, (7.5). It is given by `r` and `s`. If `C=A(p,q)`, the `k`th block consists of rows `r[k]` through `r[k+1]-1` and columns `s[k]` through `s[k+1]-1` of `C`. The first block is the rectangular matrix (7.7) and the last block is (7.8) if they are not 0-by-0. The middle blocks are the strongly connected components of A_{23}. Note that (7.7) can have columns but no rows (A_{11} is 0-by-`cc[1]` and A_{12} is 0-by-0). Similarly, (7.8) can have rows but no columns.

Maximum matching: `cs_dmperm` finds the maximum matching `jmatch[i]=j` and its inverse `imatch[j]=i`.

Coarse decomposition: A breadth-first search starting from unmatched columns $\overline{\mathcal{C}}$ finds $\mathcal{C}_1$ and $\mathcal{R}_1$. Using the matrix A^T, another breadth-first search starting from rows in $\overline{\mathcal{R}}$ determines $\mathcal{C}_3$ and $\mathcal{R}_3$. In the code, $\overline{\mathcal{C}}$ and $\overline{\mathcal{R}}$ are `C0` and `R0`, respectively. At this point, the coarse decomposition is determined solely by the flag arrays `wi` and `wj` and the matching `jmatch` and `imatch`. These arrays are scanned to find the permutations `p` and `q` and the coarse set sizes `cc` and `rr`.

Fine decomposition: The strongly connected components of the `A(R2,C2)` submatrix are found. The `A(R2,C2)` matrix is formed by first computing `C=A(p,q)` and then removing all rows and columns not in the set `R2` or `C2`. The `cs_fkeep` function is used to drop entries not in `R2`, and then the size of `R1` is subtracted from all row indices. The strongly connected components could be found without forming `C=A(R2,C2)` explicitly, but this method results in a simpler `cs_scc` function.

Combine decompositions: The fine and coarse decompositions are combined into the `r` and `s` vectors that determine the boundaries of the blocks. The permutation vectors `p` and `q` from the coarse decomposition are combined with the permutation `scc->p` of `A(R2,C2)` that reveals its strongly connected components.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
csd *cs_dmperm (const cs *A, int seed)
{
    int m, n, i, j, k, cnz, nc, *jmatch, *imatch, *wi, *wj, *pinv, *Cp, *Ci,
        *ps, *rs, nb1, nb2, *p, *q, *cc, *rr, *r, *s, ok ;
    cs *C ;
    csd *D, *scc ;
    /* --- Maximum matching ------------------------------------------------ */
    if (!CS_CSC (A)) return (NULL) ;            /* check inputs */
    m = A->m ; n = A->n ;
    D = cs_dalloc (m, n) ;                      /* allocate result */
    if (!D) return (NULL) ;
    p = D->p ; q = D->q ; r = D->r ; s = D->s ; cc = D->cc ; rr = D->rr ;
    jmatch = cs_maxtrans (A, seed) ;            /* max transversal */
    imatch = jmatch + m ;                       /* imatch = inverse of jmatch */
    if (!jmatch) return (cs_ddone (D, NULL, jmatch, 0)) ;
    /* --- Coarse decomposition -------------------------------------------- */
    wi = r ; wj = s ;                           /* use r and s as workspace */
    for (j = 0 ; j < n ; j++) wj [j] = -1 ;     /* unmark all cols for bfs */
    for (i = 0 ; i < m ; i++) wi [i] = -1 ;     /* unmark all rows for bfs */
    cs_bfs (A, n, wi, wj, q, imatch, jmatch, 1) ;       /* find C1, R1 from C0*/
    ok = cs_bfs (A, m, wj, wi, p, jmatch, imatch, 3) ;  /* find R3, C3 from R0*/
    if (!ok) return (cs_ddone (D, NULL, jmatch, 0)) ;
    cs_unmatched (n, wj, q, cc, 0) ;                    /* unmatched set C0 */
    cs_matched (n, wj, imatch, p, q, cc, rr, 1, 1) ;    /* set R1 and C1 */
    cs_matched (n, wj, imatch, p, q, cc, rr, 2, -1) ;   /* set R2 and C2 */
    cs_matched (n, wj, imatch, p, q, cc, rr, 3, 3) ;    /* set R3 and C3 */
    cs_unmatched (m, wi, p, rr, 3) ;                    /* unmatched set R0 */
    cs_free (jmatch) ;
    /* --- Fine decomposition ---------------------------------------------- */
    pinv = cs_pinv (p, m) ;         /* pinv=p' */
    if (!pinv) return (cs_ddone (D, NULL, NULL, 0)) ;
    C = cs_permute (A, pinv, q, 0) ;/* C=A(p,q) (it will hold A(R2,C2)) */
    cs_free (pinv) ;
    if (!C) return (cs_ddone (D, NULL, NULL, 0)) ;
    Cp = C->p ;
    nc = cc [3] - cc [2] ;          /* delete cols C0, C1, and C3 from C */
    if (cc [2] > 0) for (j = cc [2] ; j <= cc [3] ; j++) Cp [j-cc[2]] = Cp [j] ;
    C->n = nc ;
    if (rr [2] - rr [1] < m)        /* delete rows R0, R1, and R3 from C */
    {
        cs_fkeep (C, cs_rprune, rr) ;
        cnz = Cp [nc] ;
        Ci = C->i ;
        if (rr [1] > 0) for (k = 0 ; k < cnz ; k++) Ci [k] -= rr [1] ;
    }
    C->m = nc ;
    scc = cs_scc (C) ;              /* find strongly connected components of C*/
    if (!scc) return (cs_ddone (D, C, NULL, 0)) ;
    /* --- Combine coarse and fine decompositions -------------------------- */
    ps = scc->p ;                   /* C(ps,ps) is the permuted matrix */
    rs = scc->r ;                   /* kth block is rs[k]..rs[k+1]-1 */
    nb1 = scc->nb  ;                /* # of blocks of A(R2,C2) */
    for (k = 0 ; k < nc ; k++) wj [k] = q [ps [k] + cc [2]] ;
    for (k = 0 ; k < nc ; k++) q [k + cc [2]] = wj [k] ;
    for (k = 0 ; k < nc ; k++) wi [k] = p [ps [k] + rr [1]] ;
    for (k = 0 ; k < nc ; k++) p [k + rr [1]] = wi [k] ;
    nb2 = 0 ;                       /* create the fine block partitions */
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
    r [0] = s [0] = 0 ;
    if (cc [2] > 0) nb2++ ;         /* leading coarse block A (R1, [C0 C1]) */
    for (k = 0 ; k < nb1 ; k++)     /* coarse block A (R2,C2) */
    {
        r [nb2] = rs [k] + rr [1] ; /* A (R2,C2) splits into nb1 fine blocks */
        s [nb2] = rs [k] + cc [2] ;
        nb2++ ;
    }
    if (rr [2] < m)
    {
        r [nb2] = rr [2] ;          /* trailing coarse block A ([R3 R0], C3) */
        s [nb2] = cc [3] ;
        nb2++ ;
    }
    r [nb2] = m ;
    s [nb2] = n ;
    D->nb = nb2 ;
    cs_dfree (scc) ;
    return (cs_ddone (D, C, NULL, 1)) ;
}
```

The breadth-first search is performed by the `cs_bfs` function below.

```
static int cs_bfs (const cs *A, int n, int *wi, int *wj, int *queue,
    const int *imatch, const int *jmatch, int mark)
{
    int *Ap, *Ai, head = 0, tail = 0, j, i, p, j2 ;
    cs *C ;
    for (j = 0 ; j < n ; j++)           /* place all unmatched nodes in queue */
    {
        if (imatch [j] >= 0) continue ; /* skip j if matched */
        wj [j] = 0 ;                    /* j in set C0 (R0 if transpose) */
        queue [tail++] = j ;            /* place unmatched col j in queue */
    }
    if (tail == 0) return (1) ;         /* quick return if no unmatched nodes */
    C = (mark == 1) ? ((cs *) A) : cs_transpose (A, 0) ;
    if (!C) return (0) ;                /* bfs of C=A' to find R3,C3 from R0 */
    Ap = C->p ; Ai = C->i ;
    while (head < tail)                 /* while queue is not empty */
    {
        j = queue [head++] ;            /* get the head of the queue */
        for (p = Ap [j] ; p < Ap [j+1] ; p++)
        {
            i = Ai [p] ;
            if (wi [i] >= 0) continue ; /* skip if i is marked */
            wi [i] = mark ;             /* i in set R1 (C3 if transpose) */
            j2 = jmatch [i] ;           /* traverse alternating path to j2 */
            if (wj [j2] >= 0) continue ;/* skip j2 if it is marked */
            wj [j2] = mark ;            /* j2 in set C1 (R3 if transpose) */
            queue [tail++] = j2 ;       /* add j2 to queue */
        }
    }
    if (mark != 1) cs_spfree (C) ;      /* free A' if it was created */
    return (1) ;
}
```

To find $\mathcal{R}_1$ and $\mathcal{C}_1$, `cs_bfs` starts at unmatched column nodes in $\overline{\mathcal{C}}$ and traverses alternating paths, according to the maximum matching found by `cs_maxtrans`.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

The order of the nodes in the sets $\mathcal{R}_1$ and $\mathcal{C}_1$ is not important, so a simpler breadth-first search can be used instead of a more complicated depth-first search. To find $\mathcal{R}_3$ and $\mathcal{C}_3$, it starts at unmatched row nodes in $\overline{\mathcal{R}}$ and searches the transpose of the graph of A. The `queue` array is workspace for the breadth-first queue. `cs_dmperm` passes `p` and `q` to `cs_bfs` to use as workspace for the breadth-first search `queue`.

`cs_matched` constructs the portions of the output permutations corresponding to the matched submatrices (`A ([R1 R2 R3], [C1 C2 C3])`).

```
static void cs_matched (int n, const int *wj, const int *imatch, int *p, int *q,
    int *cc, int *rr, int set, int mark)
{
    int kc = cc [set], j ;
    int kr = rr [set-1] ;
    for (j = 0 ; j < n ; j++)
    {
        if (wj [j] != mark) continue ;      /* skip if j is not in C set */
        p [kr++] = imatch [j] ;
        q [kc++] = j ;
    }
    cc [set+1] = kc ;
    rr [set] = kr ;
}
```

`cs_unmatched` constructs the `R0` and `C0` sets. The `cs_rprune` function is used with `cs_fkeep` to remove all rows not in the set `R2`.

```
static void cs_unmatched (int m, const int *wi, int *p, int *rr, int set)
{
    int i, kr = rr [set] ;
    for (i = 0 ; i < m ; i++) if (wi [i] == 0) p [kr++] = i ;
    rr [set+1] = kr ;
}

static int cs_rprune (int i, int j, double aij, void *other)
{
    int *rr = (int *) other ;
    return (i >= rr [1] && i < rr [2]) ;
}
```

In MATLAB, the corresponding decomposition is `[p,q,r,s]=dmperm(A)`. The four outputs of `dmperm` are identical to the same outputs of `cs_dmperm`. The `dmperm` function in MATLAB 7.2 does not return the coarse decomposition. The matrix `A([R1 R2 R3], [C1 C2 C3])` permuted by `cs_dmperm` has a zero-free diagonal of maximum size. This is almost the same as `dmperm`, except that `dmperm` returns the sets `C0` and `C1` jumbled together in no particular order, as are the sets `R3` and `R0`. The function `cs_dmperm` makes the distinction between these sets clearer by returning the maximum matching as the diagonal of the square matrix `A([R1 R2 R3], [C1 C2 C3])`. Also, `dmperm` in MATLAB does not explicitly return the coarse decomposition (`cc` and `rr`). `cs_dmperm` returns the columns of each block in natural order, while `dmperm` does not. The two functions may not find the same matching, since any maximum matching is valid. Both functions return a valid Dulmage–Mendelsohn decomposition.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

7.5 Bandwidth and profile reduction

The *profile* (also called the *envelope*) of a symmetric matrix is a measure of how close its entries are to the diagonal. Assuming the diagonal of A is nonzero, the profile of A is $\sum_{j=1}^{n}(j - \min \mathcal{A}_{*j})$. The bandwidth of the matrix is similar, $\max_j(j - \min \mathcal{A}_{*j})$. A permutation that reduces the profile or bandwidth of a matrix can provide a good ordering for the LU or Cholesky factorization of PAP^T. Minimizing the profile or bandwidth is NP-hard, so heuristics are used.

The reverse Cuthill–McKee algorithm is one of the simplest heuristics. It starts at a given node and orders it first. Next, all unnumbered neighbors of any ordered nodes are ordered by increasing degree. This gives the Cuthill–Mckee ordering; the profile is reduced further by reversing this ordering (the profile cannot be increased by this reversal). Finding a good starting node is critical for obtaining a good ordering. The method starts with a *pseudoperipheral node*, which is found by repeated breadth-first searches. An arbitrary starting node is selected, and a breadth-first search determines the most distant node from this starting node. The breadth-first search is repeated, starting at this distant node, and the process repeats until the distance does not increase. The pseudoperipheral node is the last node found, and becomes the starting node for the reverse Cuthill–McKee algorithm. The MATLAB function for reverse Cuthill–McKee is `p=symrcm(A)`. CSparse does not include a C version of this algorithm.

The *Fiedler vector* of a symmetric matrix A is the eigenvector corresponding to the second largest eigenvalue of the Laplacian of the graph of A (assuming the graph is connected). The Laplacian S has the same nonzero pattern as A. Off-diagonal entries are replaced with -1, and the kth diagonal is replaced by the number of off-diagonal entries in the kth column. With the permutation P obtained from sorting the Fiedler vector, the matrix PAP^T will tend to have a small profile. It can be computed using the MATLAB `eigs` function. The `eigs` function uses a shift-and-invert technique; it thus computes the factorization of the positive semidefinite matrix $S - \alpha I$. For the Fiedler vector, the Lanczos method could be used instead, which does not require the factorization of S.

```
function [p,v,d] = cs_fiedler (A)
%CS_FIEDLER the Fiedler vector of a connected graph.
%   [p,v,d] = cs_fiedler(A) computes the Fiedler vector v (the eigenvector
%   corresponding to the 2nd smallest eigenvalue d of the Laplacian of the graph
%   of A+A').  p is the permutation obtained when v is sorted.  A should be a
%   connected graph.
%
%   See also CS_SCC, EIGS, SYMRCM, UNMESH.

n = size (A,1) ;
if (n < 2) p = 1 ; v = 1 ; d = 0 ; return ; end
opt.disp = 0 ;                      % turn off printing in eigs
opt.tol = sqrt (eps) ;
S = A | A' | speye (n) ;            % compute the Laplacian of A
S = diag (sum (S)) - S ;
[v,d] = eigs (S, 2, 'SA', opt) ;    % find the Fiedler vector v
v = v (:,2) ;
d = d (2,2) ;
[ignore p] = sort (v) ;             % sort it to get p
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

7.6 Nested dissection

Nested dissection is a fill-reducing ordering well suited to matrices arising from the discretization of a problem with two-dimensional (2D) or three-dimensional (3D) geometry. The goal of this ordering is the same as the minimum degree ordering; it is a heuristic for reducing fill-in, not the profile or bandwidth. Consider the undirected graph of a matrix A with symmetric nonzero pattern. Nested dissection finds a *node separator* that splits the graph into two or more roughly equal-sized subgraphs (left and right) when the nodes in the separator (and their incident edges) are removed from the graph. The subgraphs are then ordered recursively via nested dissection for a large subgraph or minimum degree for a small one.

With a single node separator, a matrix is split into the following form, where A_{33} is the matrix corresponding to the nodes in the node separator, A_{11} corresponds to the left subgraph, and A_{22} corresponds to the right subgraph. Since the left subgraph (A_{11}) and right subgraph (A_{22}) are not joined by any edges, A_{12} is zero.

$$\begin{bmatrix} A_{11} & & A_{13} \\ & A_{22} & A_{23} \\ A_{13}^T & A_{23}^T & A_{33} \end{bmatrix}.$$

There are many methods for finding a good node separator. One class of methods starts with an *edge separator* and then converts it into a node separator. Likewise, an edge separator can be found in many ways; the method discussed here is based on the profile-reducing methods discussed in the previous section. There are many ways of finding a nested dissection ordering; this method was chosen for its simplicity of implementation. State-of-the-art methods are highlighted at the end of this section. See also Section 7.7.

Suppose a profile-reducing ordering P has been found. Divide the matrix PAP^T into its first $\lfloor n/2 \rfloor$ rows and columns and its last $n - \lfloor n/2 \rfloor$ rows and columns.

$$PAP^T = \begin{bmatrix} A_{11} & A_{12} \\ A_{12}^T & A_{22} \end{bmatrix}.$$

Since the profile of PAP^T has been reduced, the number of entries in A_{12} will be small. If these entries (edges in the graph G) are removed, the graph splits into two components of equal size (possibly more than two connected components if the graphs of A_{11} or A_{22} are unconnected). The edges in A_{12} are an edge separator of G. In practice, the matrix is not divided equally, since the size of the edge separator can often be reduced if the two subgraphs are allowed to differ in size (they are kept roughly equal in size; otherwise, a good ordering is not obtained). The `cs_esep` M-file shown below finds an edge separator using `symrcm`.

The Dulmage–Mendelsohn decomposition can convert this edge separator into a node separator by finding a minimal *node cover* of the edges in A_{12}. Consider the

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

bipartite graph of $S = A_{12}$ and its Dulmage–Mendelsohn decomposition,

$$S = \begin{array}{c} \begin{array}{cccc} \overline{\mathcal{C}} & \mathcal{C}_1 & \mathcal{C}_2 & \mathcal{C}_3 \end{array} \\ \left[\begin{array}{cccc} S_{11} & S_{12} & S_{13} & S_{14} \\ & & S_{23} & S_{24} \\ & & & S_{34} \\ & & & S_{44} \end{array}\right] \end{array} \begin{array}{c} \mathcal{R}_1 \\ \mathcal{R}_2 \\ \mathcal{R}_3 \\ \overline{\mathcal{R}} \end{array} .$$

The method can select as the node cover either the set $\mathcal{R}_1 \cup \mathcal{C}_2 \cup \mathcal{C}_3$ or $\mathcal{R}_1 \cup \mathcal{R}_2 \cup \mathcal{C}_3$, both with size equal to the size of the maximal matching of A_{12} (its structural rank). Any edge in S will be incident on at least one node in one of these two sets. The `cs_sep` M-file selects $\mathcal{R}_1 \cup \mathcal{C}_2 \cup \mathcal{C}_3$. In practice, the set is chosen that best balances the sizes of the left and right subgraphs. The `cs_nsep` M-file constructs an edge separator and converts it into a node separator. The recursive `cs_nd` M-file finds a node separator using `cs_nsep` and then recursively bisects the two subgraphs. Small graphs (of order 500 or less) are ordered with `cs_amd`.

```
function [a,b] = cs_esep (A)
%CS_ESEP find an edge separator of a symmetric matrix A
%   [a,b] = cs_esep(A) finds a edge separator s that splits the graph of A
%   into two parts a and b of roughly equal size.  The edge separator is the
%   set of entries in A(a,b).
%
%   See also CS_NSEP, CS_SEP, CS_ND, SYMRCM.

p = symrcm (A) ;
n2 = fix (size(A,1)/2) ;
a = p (1:n2) ;
b = p (n2+1:end) ;
```

```
function [s,as,bs] = cs_sep (A,a,b)
%CS_SEP convert an edge separator into a node separator.
%   [s,as,bs] = cs_sep (A,a,b) converts an edge separator into a node separator.
%   [a b] is a partition of 1:n, thus the edges in A(a,b) are an edge separator
%   of A.  s is the node separator, consisting of a node cover of the edges of
%   A(a,b).  as and bs are the sets a and b with s removed.
%
%   See also CS_DMPERM, CS_NSEP, CS_ESEP, CS_ND.

[p q r s cc rr] = cs_dmperm (A (a,b)) ;
s = [(a (p (1:rr(2)-1))) (b (q (cc(3):(cc(5)-1))))] ;
w = ones (1, size (A,1)) ;
w (s) = 0 ;
as = a (find (w (a))) ;
bs = b (find (w (b))) ;
```

```
function [s,a,b] = cs_nsep (A)
%CS_NSEP find a node separator of a symmetric matrix A.
%   [s,a,b] = cs_nsep(A) finds a node separator s that splits the graph of A
%   into two parts a and b of roughly equal size.  If A is unsymmetric, use
%   cs_nsep(A|A').  The permutation p = [a b s] is a one-level dissection of A.
%
%   See also CS_SEP, CS_ESEP, CS_ND.

[a b] = cs_esep (A) ;
[s a b] = cs_sep (A, a, b) ;
```

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

```
function p = cs_nd (A)
%CS_ND generalized nested dissection ordering.
%   p = cs_nd(A) computes the nested dissection ordering of a matrix.  Small
%   submatrices (order 500 or less) are ordered via cs_amd.  A must be sparse
%   and symmetric (use p = cs_nd(A|A') if it is not symmetric).
%
%   See also CS_AMD, CS_SEP, CS_ESEP, CS_NSEP, AMD.

n = size (A,1) ;
if (n == 1)
    p = 1 ;
elseif (n < 500)
    p = cs_amd (A) ;                % use cs_amd on small graphs
else
    [s a b] = cs_nsep (A) ;         % find a node separator
    a = a (cs_nd (A (a,a))) ;       % order A(a,a) recursively
    b = b (cs_nd (A (b,b))) ;       % order A(b,b) recursively
    p = [a b s] ;                   % concatenate to obtain the final ordering
end
```

The Fiedler vector, or other eigenvector techniques, can lead to a smaller node separator and orderings with lower fill-in, but they are prohibitively expensive to compute for large graphs. To overcome this problem, the graph G of A can be successively *coarsened*. A node in the coarse graph G_c of A represents a unique set of nodes in G with node weight equal to the number of nodes it represents. Edge weights are used to reflect the number of edges (the sum of their weights) between sets of nodes in G. A sequence of coarser and coarser graphs G (the original graph), $G_1, G_2, \ldots, G_k$ is found until G_k is small enough to use powerful edge or node separator methods efficiently. Next, the edge or node separator is mapped to the graph of G_{k-1}, and refinement techniques (such as the Kernighan–Lin algorithm) are used to improve this partition of G_{k-1}. The refinement process continues until a separator of G is obtained.

Figure 7.4 is the graph of the matrix in Figure 4.2 on page 39 with both node and edge separators highlighted.

If applied to a 2D s-by-s mesh with node separators selected along a mesh line that most evenly divides the graph, nested dissection leads to an asymptotically optimal ordering, with $31(n \log_2 n)/8 + O(n)$ nonzeros in L, and requiring $829(n^{3/2})/84 + O(n \log n)$ floating-point operations to compute, where $n = s^2$ is the dimension of the matrix. For a 3D s-by-s-by-s mesh the dimension of the matrix is $n = s^3$. There are $O(n^{4/3})$ nonzeros in L, and $O(n^2)$ floating-point operations are required to compute L, which is also asymptotically optimal.

7.7 Further reading

Finding a good fill-reducing ordering is essential for reducing time and memory requirements for sparse factorization methods. Since it is an NP-hard problem (Yannakakis [199]), many heuristic methods have been developed. These heuristics can be divided into three broad categories: minimum degree, nested dissection, and profile reduction. The Dulmage–Mendelsohn decomposition [72] is not a heuristic, but it can be considered as a fourth category of ordering methods, since it restricts

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

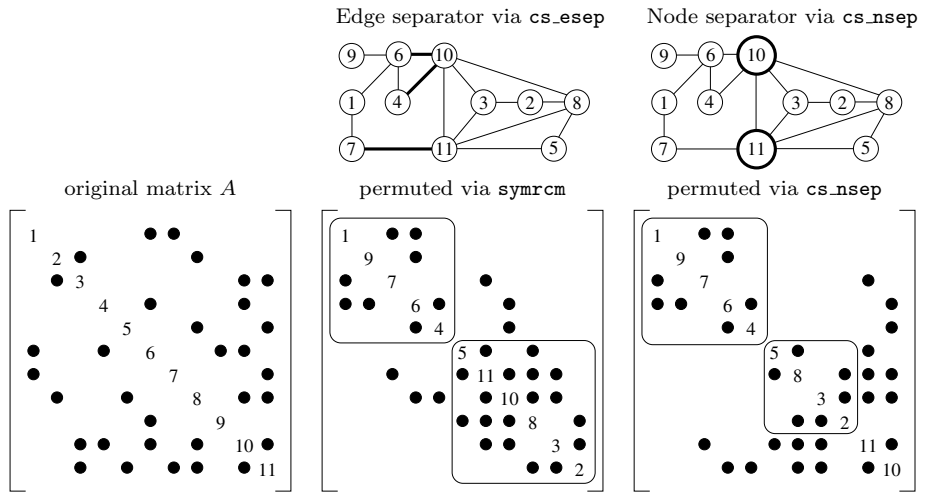


Figure 7.4. *A node and edge separator*

fill-in to the diagonal blocks.

Minimum degree and its variants are the most commonly used methods, providing low fill-in on the widest range of matrices. Good nested dissection orderings tend to be superior to minimum degree for large problems that come from 2D and 3D spatial discretizations (finite-element methods, for example), particularly when coupled with minimum degree for ordering small subgraphs. An early unsymmetric form is Markowitz's method [155] that selects a_{ij} to minimize the product of the row degree i and column degree j. MA28 by Duff and Reid [61] in the HSL package is based on an efficient implementation of the method. It is interesting to note that Markowitz received the 1990 Nobel prize [79] in economics for his pioneering work in portfolio theory, part of which is his LU ordering work.[14]

Minimum degree is a local greedy strategy, or a "bottom-up" approach, since it finds the leaves of the elimination tree first. Tinney and Walker [196] developed the first minimum degree method for symmetric matrices. Note that the word "optimal" in the title of their paper is a misnomer, since minimum degree is a non-optimal method. Its earliest efficient implementation is by George and Liu's algorithm (MMD and its precursors in SPARSPAK [85, 87, 91]). Other implementations include YSMP (Eisenstat et al. [73] and Eisenstat, Schultz, and Sherman [76]) and MA27 (Duff, Erisman, and Reid [53] and Duff and Reid [62]).

Since computing the exact degree is costly, Amestoy, Davis, and Duff developed AMD, an approximate minimum degree ordering [1, 2]; Davis et al. later extended this to compute a column ordering of A^TA without forming A^TA explicitly (COLAMD [33, 34]). The approximate degree spurred the search for approximations to the *deficiency*. The deficiency of a node is how much fill-in would occur if

[14] http://nobelprize.org/economics/laureates/1990/markowitz-autobio.html

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

the node were selected as the pivot. Methods based on variations of approximate deficiency have been developed by Rothberg and Eisenstat [176], Ng and Raghavan [161], and Pellegrini, Roman, and Amestoy [166].

Nested dissection is a "top-down" approach, since the first level separator includes the root of the elimination tree and its immediate descendants. Kernighan and Lin [140] present an early graph partitioning technique based on exchanging pairs of nodes in the graph. Fiduccia and Mattheyses present a more efficient node-swapping method [77]. Hager, Park, and Davis extend this idea to exchange blocks of nodes [126]. The first nested dissection algorithm for ordering sparse matrices is due to George [81, 82]. It relies on finding a good pseudoperipheral node, as discussed by George and Liu [86]. See also Duff, Erisman, and Reid's discussion of George's nested dissection method [52].

More recent approaches to nested dissection are based on multilevel methods and eigenvector techniques (in particular the Fiedler vector [78]). These include methods by Pothen, Simon, and Liou [170], Karypis and Kumar (METIS [139]), Hendrickson and Leland (CHACO [131]), Pellegrini, Roman, and Amestoy (SCOTCH [166]), and Walshaw, Cross, and Everett (JOSTLE [197]). Since many matrices arise in problems with 2D and 3D geometry, Heath and Raghavan [129] and Gilbert, Miller, and Teng (MESHPART [104]) present partitioning methods based on the geometric position of nodes in the graph. CHOLMOD includes both AMD and a partitioning method that combines METIS with a constrained approximate column minimum degree ordering algorithm, CCOLAMD [30].

Many of the early sparse matrix factorization methods used a profile or envelope data structure, so reducing the profile of a matrix had a direct impact on the memory usage of the method. Profile reduction is still a useful method for more recent factorization techniques. It can form a first step in finding a good edge or node separator for graph partitioning and nested dissection. Cuthill and McKee [25] developed one of the first techniques, which is still in use. Liu and Sherman [153] showed that reversing the Cuthill–McKee ordering never increases the profile and often reduces it. Chan and George [21] present an efficient implementation. Other profile reduction techniques include those by Crane et al. [24], Gibbs [99], Gibbs, Poole, and Stockmeyer [100], Hager [125], Reid and Scott [172, 173], Lewis [146], and Sloan [187]. Eigenvector techniques are also an effective method for reducing the profile, as discussed by Barnard, Pothen, and Simon [14], Pothen, Simon, and Liou [170], and Kumfert and Pothen [142].

Cormen, Leiserson, and Rivest [23] describe the *scc* algorithm discussed in Section 7.3. The earliest algorithm for finding the strongly connected components of a graph is due to Tarjan [194]; Duff and Reid [59, 60] implement the algorithm. Gustavson [120] discusses both the maximum matching and an implementation of Tarjan's algorithm. Duff [48, 49] presents the $O(|A|n)$-time maximum matching algorithm used by `cs_maxtrans`. Duff and Wiberg [71] implement an $O(|A|\sqrt{n})$-time maximum matching algorithm of Hopcroft and Karp [137] that is not always faster than the $O(|A|n)$-time algorithm in practice. Pothen and Fan [169] compare various methods for computing the block triangular form. Duff and Koster [57, 58] present a maximum weighted matching.

Ordering methods in MATLAB are discussed in Chapter 10.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Exercises

7.1. Download a copy of AMD from www.siam.org/books/fa02 (or from www.acm.org as ACM Algorithm 837). Compare it with `cs_amd` and make a list of the differences between the two codes. Compare the run time, memory usage, and ordering quality on a large range of symmetric matrices (use `p=cs_amd(A)` and compare with `p=amd(A)` in MATLAB). The MATLAB expression `lnz=sum(symbfact(A(p,p)))` gives the number of nonzeros in the Cholesky factor L of the matrix `A(p,p)` (ignoring numerical cancellation).

7.2. Compare the MATLAB statement `q=cs_amd(A,2)` with the permutation computed by the MATLAB statement `q=colamd(A)`. Compare the ordering time and memory usage. Use the column ordering in `[L,U,P]=lu(A(:,q))` to compare the ordering quality (`nnz(L)+nnz(U)`). Add code to `cs_amd` and COLAMD to compute their memory usage. COLAMD orders A^TA without forming it explicitly, so it will tend to use much less memory than `cs_amd(A)`. Both drop dense rows from `A`.

7.3. Compare the MATLAB statement `p=cs_amd(A,3)` with the permutation computed by the MATLAB statement `p=amd(A'*A)` (see Problem 7.1). Compare the ordering time and memory usage. Compare ordering quality; use `rnz =sum(symbfact(A(:,q),'col'))` (where `rnz` is the same as `nnz(qr(A,0))`, ignoring numerical cancellation and assuming `A` is not structurally rank deficient).

7.4. Write a function that solves $Ax = b$ by combining LU factorization with the block triangular form (the fine Dulmage–Mendelsohn decomposition). Find the blocks with `cs_dmperm` and then analyze and factorize each block with `cs_sqr` and `cs_lu`, respectively. Next, solve $Ax = b$ via a block backsolve. Compare with `cs_lu` on matrices with many diagonal blocks (these include matrices arising in circuit simulation and chemical process simulation). See also Section 8.4.

7.5. Why is the block triangular form not helpful for sparse Cholesky factorization? Hint: consider the elimination tree postordering. What happens if the elimination tree is a forest?

7.6. Heuristics for placing large entries on the diagonal of a matrix are useful methods for reducing the need for partial pivoting during factorization (see [57, 58], for example). Try the following method. First, scale a copy of the matrix A so that the largest entry in each column is equal to one. Next, remove small entries from the matrix and use `cs_maxtrans` to find a zero-free diagonal. If too many entries were dropped, decrease the drop tolerance and try again, or simply complete the matching arbitrarily. Use the matching as a column preordering Q and then order $AQ + (AQ)^T$ with minimum degree. Use a small pivot tolerance in `cs_lu` and determine how many off-diagonal pivots are found.

7.7. Compare the run time of `cs_dmperm` with different values of `seed` (`0`, `-1`, and `1`) on a wide range of matrices from real applications. Symmetric indefinite

matrices arising in optimization problems are of particular interest (many of the matrices from Gould, Hu, and Scott [116] in the `GHS_indef` group in the UF sparse matrix collection, for example). Find examples where the randomized order, reverse order, and natural order methods each outperform the other methods (the `boyd2` matrix is an extreme example).

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php